

SECOND SEMESTER M.Sc COMPUTER SCIENCE WITH
SPECIALIZATION IN ARTIFICIAL INTELLIGENCE

(2024 - 2026 BATCH)

LAB RECORD

CAI-CC-524 PATTERN RECOGNITION LABORATORY

*Submitted to the University of Kerala in partial fulfilment of the requirement for
the award of the degree of Master of Science in Computer Science (with
Specialization in Artificial Intelligence)*



Submitted By

Hrishikesh M Jayakrishnan

97424607010

*Department of Computer Science,
University of Kerala, Kariavattom,
Thiruvananthapuram, Kerala, 695581*

Index

Sl No	Name of the program	Page No
1	LDA for dimensionality reduction	1
2	PCA for dimensionality reduction and KNN	9
3	PCA on Iris dataset	14
4	Euclidean distance between two grayscale images	26
5	PCA vs LDA on MNIST dataset	29
6	Implementing distance metrics on image dataset	38
7	Operations on fuzzy sets and De Morgan's Law	42
8	Membership functions using NumPy	52
9	Fuzzification of input temperature values	56
10	Comparison of different defuzzification methods	59
11	Calculate value functions for a given Markov Decision Process	62

Cycle II

Experiment 1

Problem

Apply LDA for dimensionality reduction and any classifier for face recognition on the ORL dataset and analyze the performance.

Aim and Objectives

To implement LDA for dimensionality reduction and evaluate its effectiveness in improving the performance of a face recognition classifier on the ORL dataset.

Program Code

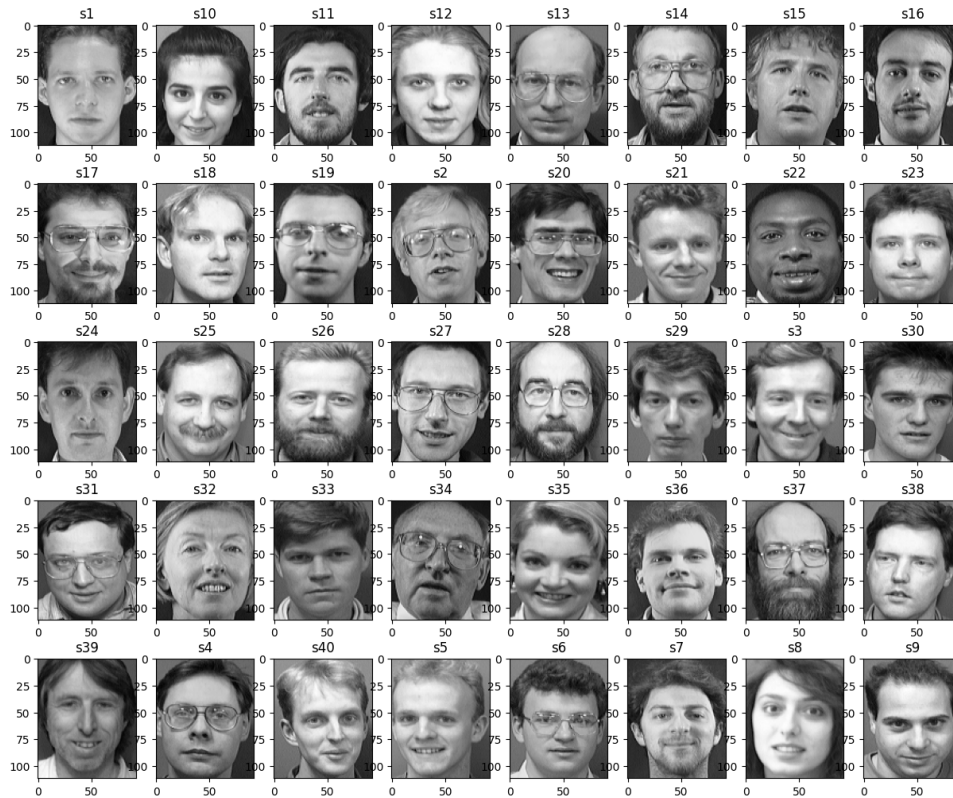
```
import os
import cv2
import matplotlib.pyplot as plt
import shutil
import pandas as pd
import numpy as np
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.discriminant_analysis import
    LinearDiscriminantAnalysis as LDA
from sklearn.metrics import accuracy_score, confusion_matrix,
    classification_report
from sklearn.svm import SVC
```

```
dataset_path = "../datasets/ORL_dataset"
```

```
classes = os.listdir(dataset_path)
```

```
plt.subplots(5, 8, figsize=(15, 12.5))
plt.suptitle("Sample Images from ORL Dataset", fontsize=12)
for class_name in classes:
    image = cv2.imread(os.path.join(dataset_path, class_name, "1.
        pgm"), cv2.IMREAD_GRAYSCALE)
    plt.subplot(5, 8, classes.index(class_name) + 1)
    plt.imshow(image, cmap='gray')
    plt.title(f"{class_name}")
```

Sample Images from ORL Dataset



```
test_ratio = 0.2

classes = os.listdir(dataset_path)

for class_name in classes:

    os.makedirs(f"../datasets/train/{class_name}", exist_ok=True)
    os.makedirs(f"../datasets/test/{class_name}", exist_ok=True)

    images = os.listdir(f"../datasets/ORL_dataset/{class_name}")
    train_images, test_images = train_test_split(images,
        test_size=test_ratio, random_state=42)

    for img in train_images:
        shutil.copy(f"../datasets/ORL_dataset/{class_name}/{img}"
            , f"../datasets/train/{class_name}/{img}")
        print(f"Copied images for {class_name} to train folder")

    for img in test_images:
        shutil.copy(f"../datasets/ORL_dataset/{class_name}/{img}"
            , f"../datasets/test/{class_name}/{img}")
        print(f"Copied images for {class_name} to test folder")
```

```
train_set = "../datasets/train"
test_set = "../datasets/test"
```

```
images_X = []
labels_y = []
for class_name in classes:
    class_path = os.path.join(train_set, class_name)
    for img_name in os.listdir(class_path):
        img_path = os.path.join(class_path, img_name)
        image = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
        images_X.append(image.flatten())
        labels_y.append(classes.index(class_name))
print(f"Number of images in train set: {len(images_X)}")
```

Number of images in train set: 320

```
labels_y = np.array(labels_y)
images_X = pd.DataFrame(images_X)
```

```
lda = LDA(n_components=39)
images_X_lda = lda.fit_transform(images_X, labels_y)
images_X_lda_df = pd.DataFrame(images_X_lda)
images_X_lda_df
```

	0	1	2	3	4	5	6	7	8	9	...
0	-4.195820	1.440183	-9.526318	0.985577	-2.238878	2.710791	-3.748485	-2.019688	0.134558	3.244815	...
1	-5.915632	3.863930	-9.567232	7.083888	-2.749441	1.659635	-1.617893	0.196132	-3.446693	2.177965	...
2	-7.663957	1.713726	-10.372344	0.992018	-2.778074	-3.278244	-0.597992	0.730577	-0.502767	2.438865	...
3	-7.147968	-0.097785	-9.072887	4.785691	-0.299194	-0.654152	-1.655769	1.404543	-0.908319	4.447840	...
4	-7.168803	0.240219	-8.515570	2.347091	0.333050	1.075298	0.154636	1.959033	-0.608113	4.345680	...
...	...	...	...	...	...	...	...	...	...	...	...
315	-0.170546	0.317235	6.252161	2.901864	-2.681779	1.557313	-0.881094	-3.414175	4.079278	0.662487	...
316	0.060807	2.557154	5.252318	2.988304	-2.362040	0.248275	0.788925	-7.416892	2.354512	2.145689	...
317	-1.398813	1.105114	7.809580	-0.047862	-3.745474	1.667271	-0.299620	-8.218174	1.227088	1.422216	...
318	-2.143815	1.692445	7.472940	1.946441	-2.147201	1.074111	0.807094	-3.373334	1.865819	1.020588	...
319	-0.624043	3.445896	6.086466	2.135989	-3.398222	1.754490	1.134245	-5.636890	1.547634	2.705579	...

320 rows × 39 columns

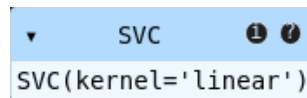
```
test_images_X = []
test_labels_y = []
for class_name in classes:
    class_path = os.path.join(test_set, class_name)
    for img_name in os.listdir(class_path):
        img_path = os.path.join(class_path, img_name)
        image = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
        test_images_X.append(image.flatten())
        test_labels_y.append(classes.index(class_name))
```

```
test_images_X_df = pd.DataFrame(test_images_X)
test_labels_y = np.array(test_labels_y)
```

```
test_images_X_lda = lda.transform(test_images_X_df)
test_images_X_lda_df = pd.DataFrame(test_images_X_lda)
```

Training SVM with LDA applied on the features

```
svm_01 = SVC(kernel="linear", gamma="scale")
svm_01.fit(images_X_lda, labels_y)
```



A screenshot of a Jupyter Notebook cell. The cell has a blue header bar with a dropdown arrow, the text 'SVC', and two circular icons (one with an 'i' and one with a question mark). Below the header, the code 'SVC(kernel='linear')' is displayed in a light blue background.

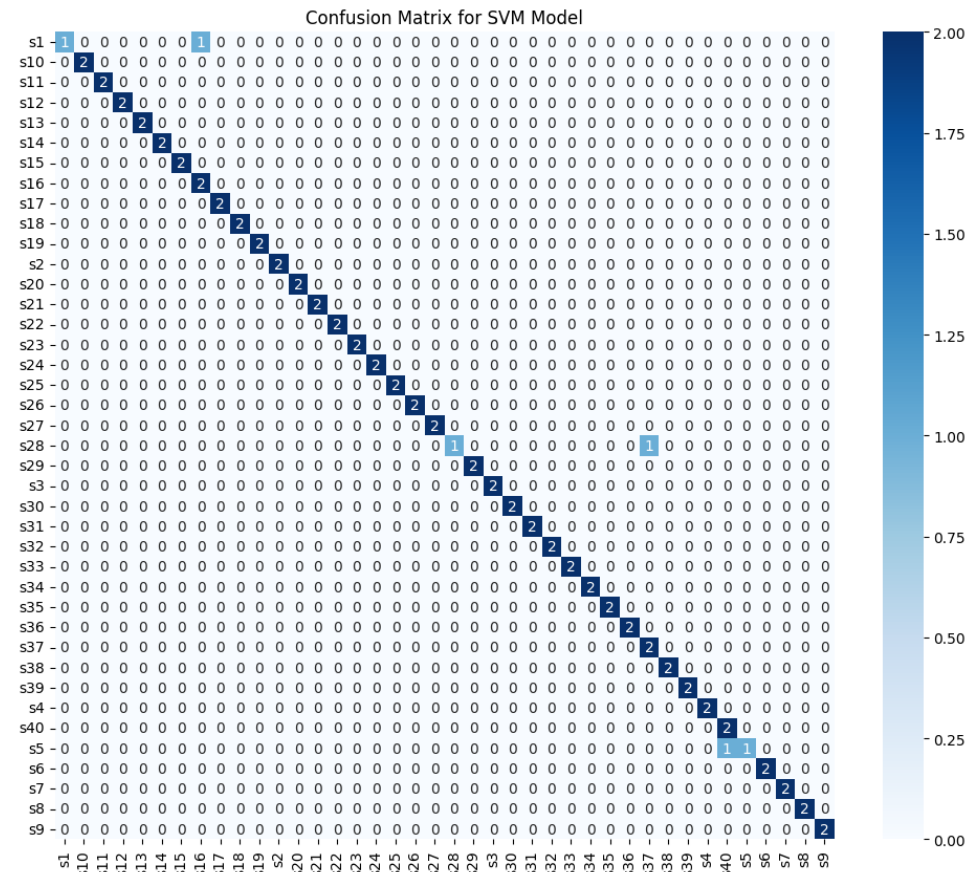
```
y_pred = svm_01.predict(test_images_X_lda)
cl_report = classification_report(test_labels_y, y_pred,
    target_names=classes)
print(cl_report)
```

	precision	recall	f1-score	support
s1	1.00	0.50	0.67	2
s10	1.00	1.00	1.00	2
s11	1.00	1.00	1.00	2
s12	1.00	1.00	1.00	2
s13	1.00	1.00	1.00	2
s14	1.00	1.00	1.00	2
s15	1.00	1.00	1.00	2
s16	0.67	1.00	0.80	2
s17	1.00	1.00	1.00	2
s18	1.00	1.00	1.00	2
s19	1.00	1.00	1.00	2
s2	1.00	1.00	1.00	2
s20	1.00	1.00	1.00	2
s21	1.00	1.00	1.00	2
s22	1.00	1.00	1.00	2
...				
accuracy			0.96	80
macro avg	0.97	0.96	0.96	80
weighted avg	0.97	0.96	0.96	80

```
ac_score = accuracy_score(test_labels_y, y_pred)
print(f"Accuracy of the SVM model on test set: {ac_score}")
```

Accuracy of the SVM model on test set: 0.9625

```
cf = confusion_matrix(test_labels_y, y_pred)
plt.figure(figsize=(12, 10))
sns.heatmap(cf, annot=True, fmt='d', cmap='Blues', xticklabels=
    classes, yticklabels=classes)
plt.title("Confusion Matrix for SVM Model")
```



Training SVM with original features

```
svm_02 = SVC(kernel="linear", gamma="scale")
svm_02.fit(images_X, labels_y)
```

[illegible]

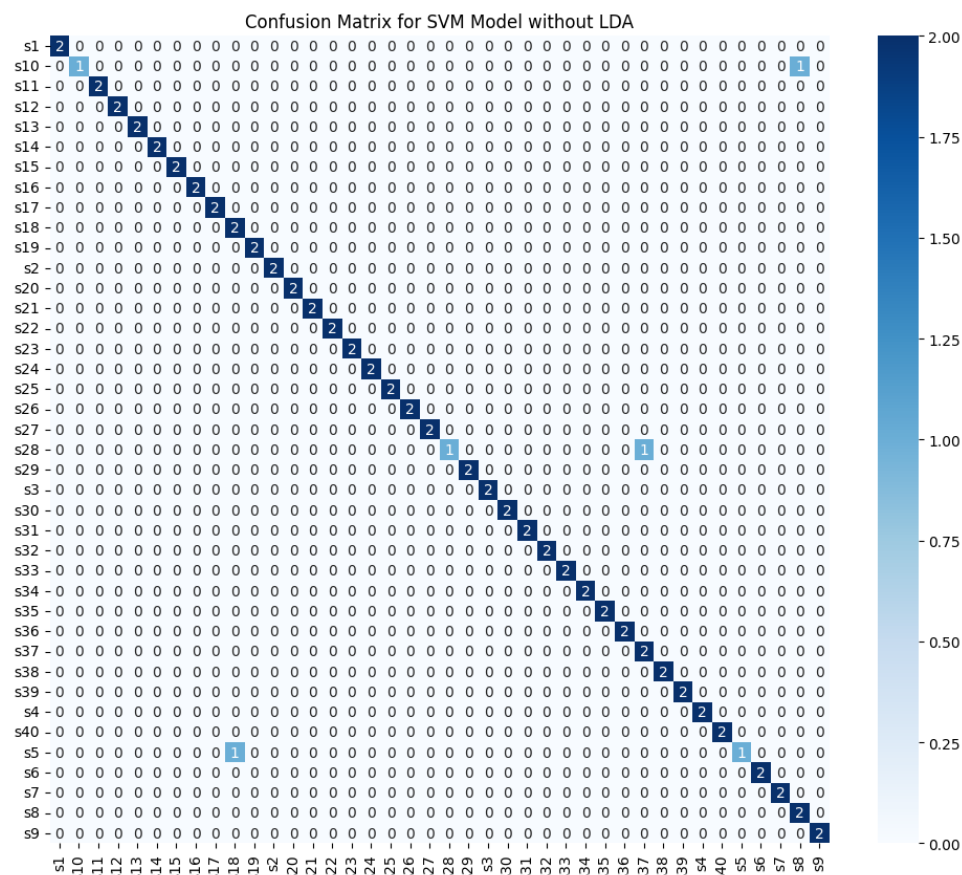
	precision	recall	f1-score	support
s1	1.00	1.00	1.00	2
s10	1.00	0.50	0.67	2
s11	1.00	1.00	1.00	2
s12	1.00	1.00	1.00	2
s13	1.00	1.00	1.00	2
s14	1.00	1.00	1.00	2
s15	1.00	1.00	1.00	2
s16	1.00	1.00	1.00	2
s17	1.00	1.00	1.00	2
s18	0.67	1.00	0.80	2
s19	1.00	1.00	1.00	2
s2	1.00	1.00	1.00	2
s20	1.00	1.00	1.00	2

s21	1.00	1.00	1.00	2
s22	1.00	1.00	1.00	2
s23	1.00	1.00	1.00	2
s24	1.00	1.00	1.00	2
s25	1.00	1.00	1.00	2
s26	1.00	1.00	1.00	2
s27	1.00	1.00	1.00	2
s28	1.00	0.50	0.67	2
s29	1.00	1.00	1.00	2
s3	1.00	1.00	1.00	2
...				
accuracy			0.96	80
macro avg	0.97	0.96	0.96	80
weighted avg	0.97	0.96	0.96	80

```
ac_score_02 = accuracy_score(test_labels_y, y_pred_02)
print(f"Accuracy of the SVM model on test set without LDA: {
    ac_score_02}")
```

Accuracy of the SVM model on test set without LDA: 0.9625

```
cf_02 = confusion_matrix(test_labels_y, y_pred_02)
plt.figure(figsize=(12, 10))
sns.heatmap(cf_02, annot=True, fmt='d', cmap='Blues', xticklabels=
    =classes, yticklabels=classes)
plt.title("Confusion Matrix for SVM Model without LDA")
```



AI-Generated

```
import numpy as np
from sklearn.datasets import fetch_olivetti_faces
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.discriminant_analysis import
    LinearDiscriminantAnalysis
from sklearn.svm import SVC
from sklearn.metrics import classification_report, accuracy_score
    , confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

# Step 1: Load the ORL Dataset
data = fetch_olivetti_faces()
X = data.data # 400 samples, 4096 features (64x64 images)
y = data.target # 40 classes (subjects)

print(f"Dataset shape: {X.shape}, Labels shape: {y.shape}")

# Step 2: Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

# Step 3: Standardize features
scaler = StandardScaler()
X_train_std = scaler.fit_transform(X_train)
X_test_std = scaler.transform(X_test)

# Step 4: Apply LDA
lda = LinearDiscriminantAnalysis(n_components=39) # max
    components = n_classes - 1
X_train_lda = lda.fit_transform(X_train_std, y_train)
X_test_lda = lda.transform(X_test_std)

print(f"LDA-transformed shape: {X_train_lda.shape}")

# Step 5: Train Classifier (SVM)
classifier = SVC(kernel='linear', C=1, random_state=42)
classifier.fit(X_train_lda, y_train)

# Step 6: Predict and Analyze Performance
y_pred = classifier.predict(X_test_lda)

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

print("Accuracy Score:", accuracy_score(y_test, y_pred))

# Step 7: Plot Confusion Matrix
```

```
conf_mat = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(12, 10))
sns.heatmap(conf_mat, annot=False, cmap='Blues', xticklabels=
    False, yticklabels=False)
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("True")
plt.show()
```

Comments on comparisons

- The AI generated code uses the built-in `fetch_olivetti_faces` function from `scikit-learn` to directly load the ORL dataset, while the code I have written manually loads images from a local directory, flattens them, and organizes them into training and testing folders. The manual approach gives more control over data preprocessing and file management.
- Both codes apply Linear Discriminant Analysis (LDA) for dimensionality reduction and use a linear SVM for classification. However, the code I have written goes a step further by also training and evaluating an SVM on the original (non-LDA) features, showing a comparison of performance with and without LDA.

Verified By

Cycle II

Experiment 2

Problem

Use PCA to reduce the dimensionality of an image dataset and apply KNN for classification

Aim and Objectives

To use PCA for reducing the dimensionality of an image dataset and apply KNN for classification, including data preprocessing, feature reduction, model training, and performance evaluation.

Program Code

```
import cupy as cp
from cuml.decomposition import PCA
from cuml.neighbors import KNeighborsClassifier
from cuml.preprocessing import LabelEncoder, StandardScaler
from cuml.metrics import accuracy_score, confusion_matrix
from sklearn.metrics import classification_report
from cuml.preprocessing import StandardScaler
import seaborn as sns
import cudf as df
import matplotlib.pyplot as plt
```

Here we are going to take the MNIST dataset

```
train_set = df.read_csv("../datasets/mnist_train.csv")
test_set = df.read_csv("../datasets/mnist_test.csv")
```

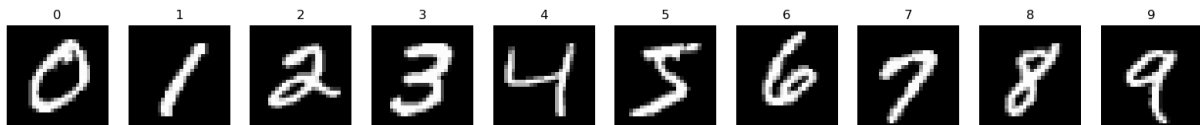
```
train_set
```

	label	1x1	1x2	1x3	1x4	1x5	1x6	1x7	1x8	1x9	...
0	5	0	0	0	0	0	0	0	0	0	...
1	0	0	0	0	0	0	0	0	0	0	...
2	4	0	0	0	0	0	0	0	0	0	...
3	1	0	0	0	0	0	0	0	0	0	...
4	9	0	0	0	0	0	0	0	0	0	...
...	...	...	...	...	...	...	...	...	...	...	...
59995	8	0	0	0	0	0	0	0	0	0	...
59996	3	0	0	0	0	0	0	0	0	0	...
59997	5	0	0	0	0	0	0	0	0	0	...
59998	6	0	0	0	0	0	0	0	0	0	...
59999	8	0	0	0	0	0	0	0	0	0	...

60000 rows × 785 columns

```
train_set.describe()
```

```
plt.subplots(1, 10, figsize=(20, 17))
for i in range(10):
    plt.subplot(1, 10, i + 1)
    image = train_set[train_set["label"] == i].to_numpy()[0][1:].
        reshape(28, 28)
    plt.imshow(image, cmap="gray")
    plt.axis("off")
    plt.title(i)
plt.show()
```



```
for i in range(10):
    print(f"Number of {i} in train set: {train_set[train_set['label'] == i].shape[0]}")
    print(f"Number of {i} in test set: {test_set[test_set['label'] == i].shape[0]}")
```

```
Number of 0 in train set: 5923
Number of 0 in test set: 980
Number of 1 in train set: 6742
Number of 1 in test set: 1135
Number of 2 in train set: 5958
Number of 2 in test set: 1032
Number of 3 in train set: 6131
Number of 3 in test set: 1010
Number of 4 in train set: 5842
Number of 4 in test set: 982
Number of 5 in train set: 5421
Number of 5 in test set: 892
Number of 6 in train set: 5918
Number of 6 in test set: 958
Number of 7 in train set: 6265
Number of 7 in test set: 1028
Number of 8 in train set: 5851
Number of 8 in test set: 974
Number of 9 in train set: 5949
Number of 9 in test set: 1009
```

```
X_train = train_set.drop("label", axis=1).to_cupy()
y_train = train_set["label"].to_cupy()
X_test = test_set.drop("label", axis=1).to_cupy()
y_test = test_set["label"].to_cupy()
```

```
std_scaler = StandardScaler()
X_train = std_scaler.fit_transform(X_train)
X_test = std_scaler.transform(X_test)
```

```
pca = PCA(n_components=39)
X_train = pca.fit_transform(X_train)
X_test = pca.transform(X_test)
```

Training a KNN classifier on the PCA applied dataset

```
knn_01 = KNeighborsClassifier(n_neighbors = 5)
knn_01.fit(X_train, y_train)
```

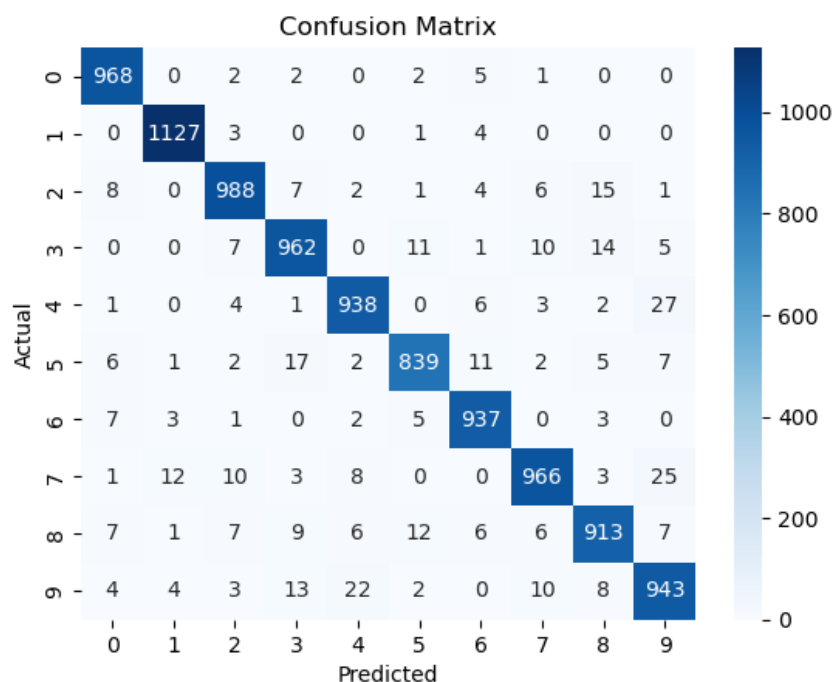
▼ KNeighborsClassifier ⓘ
KNeighborsClassifier()

```
pred_01 = knn_01.predict(X_test)
```

```
print("Accuracy:", accuracy_score(y_test, pred_01))
conf_matrix = confusion_matrix(y_test, pred_01)

sns.heatmap(conf_matrix.get(), annot=True, fmt="d", cmap="Blues",
            xticklabels=range(10), yticklabels=range(10))
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

Accuracy: 0.9581



```
print(classification_report(y_test.get(), pred_01.get()))
```

	precision	recall	f1-score	support
0	0.97	0.99	0.98	980
1	0.98	0.99	0.99	1135
2	0.96	0.96	0.96	1032
3	0.95	0.95	0.95	1010
4	0.96	0.96	0.96	982
5	0.96	0.94	0.95	892
6	0.96	0.98	0.97	958
7	0.96	0.94	0.95	1028
8	0.95	0.94	0.94	974
9	0.93	0.93	0.93	1009
accuracy			0.96	10000
macro avg	0.96	0.96	0.96	10000
weighted avg	0.96	0.96	0.96	10000

AI-Generated

```
import numpy as np
from sklearn.datasets import fetch_olivetti_faces
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.discriminant_analysis import
    LinearDiscriminantAnalysis
from sklearn.svm import SVC
from sklearn.metrics import classification_report, accuracy_score
    , confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

# Step 1: Load the ORL Dataset
data = fetch_olivetti_faces()
X = data.data # 400 samples, 4096 features (64x64 images)
y = data.target # 40 classes (subjects)

print(f"Dataset shape: {X.shape}, Labels shape: {y.shape}")

# Step 2: Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

# Step 3: Standardize features
scaler = StandardScaler()
X_train_std = scaler.fit_transform(X_train)
X_test_std = scaler.transform(X_test)

# Step 4: Apply LDA
lda = LinearDiscriminantAnalysis(n_components=39) # max
    components = n_classes - 1
```

```

X_train_lda = lda.fit_transform(X_train_std, y_train)
X_test_lda = lda.transform(X_test_std)

print(f"LDA-transformed shape: {X_train_lda.shape}")

# Step 5: Train Classifier (SVM)
classifier = SVC(kernel='linear', C=1, random_state=42)
classifier.fit(X_train_lda, y_train)

# Step 6: Predict and Analyze Performance
y_pred = classifier.predict(X_test_lda)

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

print("Accuracy Score:", accuracy_score(y_test, y_pred))

# Step 7: Plot Confusion Matrix
conf_mat = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(12, 10))
sns.heatmap(conf_mat, annot=False, cmap='Blues', xticklabels=
    False, yticklabels=False)
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("True")
plt.show()

```

Comments on comparisons

- The AI generated code has used sklearn which is a cpu based library compared to the code I have written which uses cuml, a GPU accelerated library.
- Both follow the same workflow—data preprocessing, PCA, KNN classification, and evaluation—but differ in dataset

Verified By

Cycle II

Experiment 3

Problem

Perform exploratory data analysis (EDA) on Iris dataset to understand its structure. Apply Principal Component Analysis (PCA) for dimensionality reduction. Visualize the transformed dataset and analyze the effectiveness of PCA in reducing the dimensions.

Aim and Objectives

To explore the Iris dataset and apply PCA for dimensionality reduction, followed by visualization and analysis of the transformed data to assess PCA's effectiveness.

Program Code

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix,
    classification_report
import seaborn as sns
import matplotlib.pyplot as plt
```

```
iris = pd.read_csv("../datasets/Iris.csv")
```

```
iris
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa
...	...	...	...	...	...	...
145	146	6.7	3.0	5.2	2.3	Iris-virginica
146	147	6.3	2.5	5.0	1.9	Iris-virginica
147	148	6.5	3.0	5.2	2.0	Iris-virginica
148	149	6.2	3.4	5.4	2.3	Iris-virginica
149	150	5.9	3.0	5.1	1.8	Iris-virginica

150 rows × 6 columns

```
iris.describe()
```

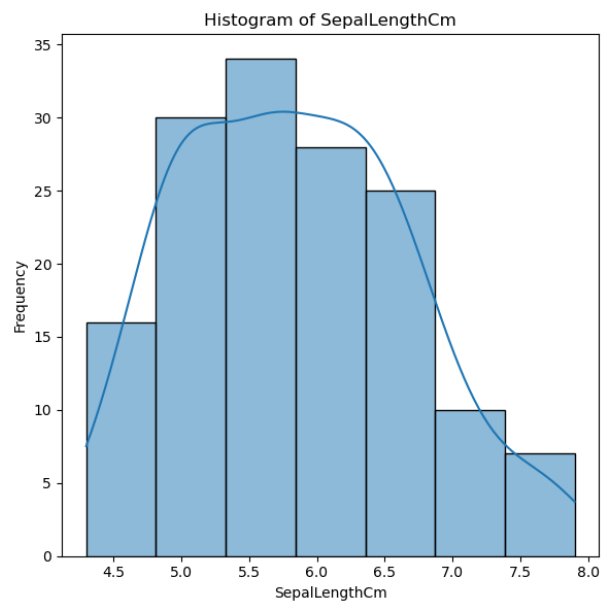

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	75.500000	5.843333	3.054000	3.758667	1.198667
std	43.445368	0.828066	0.433594	1.764420	0.763161
min	1.000000	4.300000	2.000000	1.000000	0.100000
25%	38.250000	5.100000	2.800000	1.600000	0.300000
50%	75.500000	5.800000	3.000000	4.350000	1.300000
75%	112.750000	6.400000	3.300000	5.100000	1.800000
max	150.000000	7.900000	4.400000	6.900000	2.500000

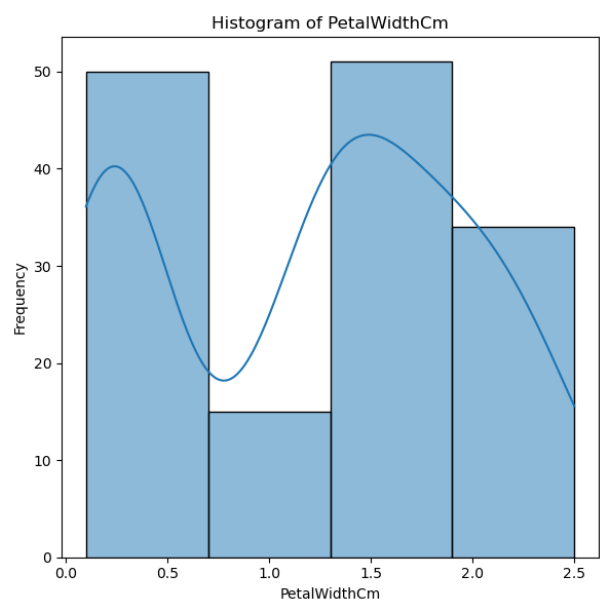
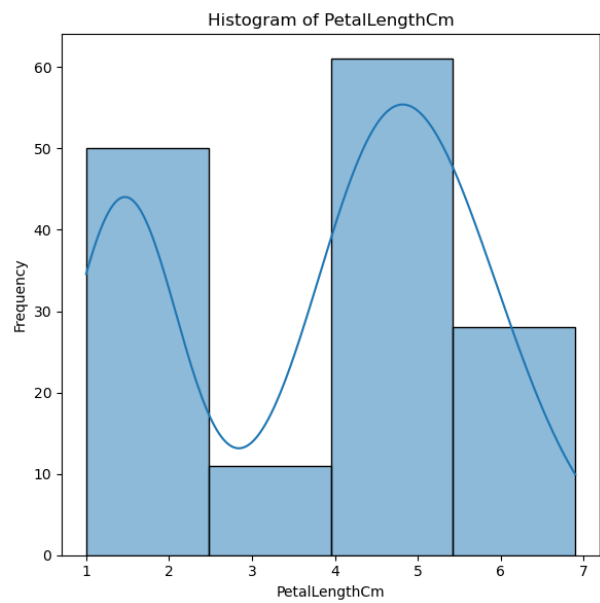
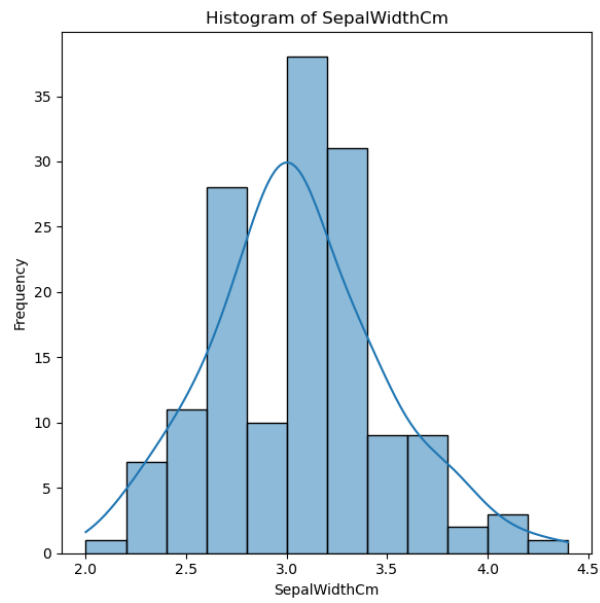
```
iris_X = iris.drop(columns=["Id", "Species"])
iris_y = iris["Species"]
```

```
for column in iris_X.columns:
    q1 = iris_X[column].quantile(0.25)
    q3 = iris_X[column].quantile(0.75)
    iqr = q3 - q1

    bin_width = 2 * iqr / (len(iris_X) ** (1 / 3))

    bins = int((iris_X[column].max() - iris_X[column].min()) /
                bin_width)
    plt.figure(figsize=(6, 6))
    sns.histplot(iris_X[column], bins=bins, kde=True)
    plt.title(f"Histogram of {column}")
    plt.xlabel(column)
    plt.ylabel("Frequency")
    plt.tight_layout()
    plt.show()
```

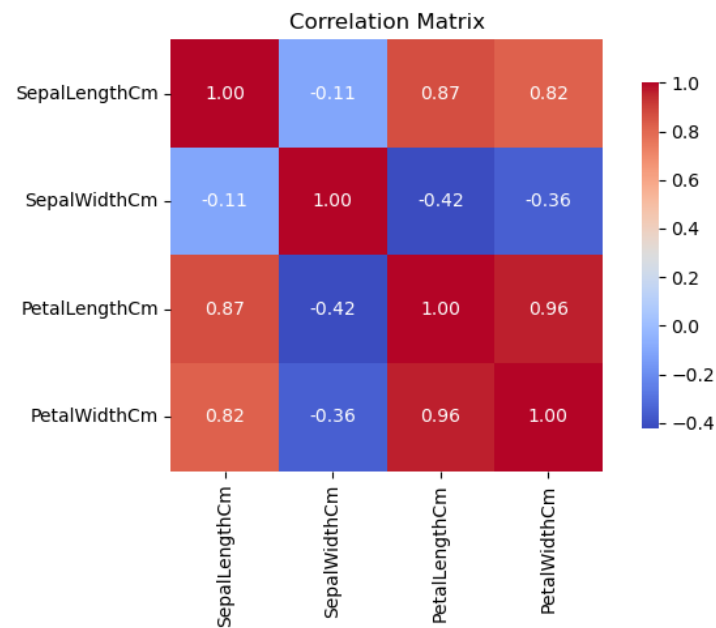




```

corr = iris_X.corr()
plt.figure(figsize=(7, 5))
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f", square=
    True, cbar_kws={"shrink": .8})
plt.title("Correlation Matrix")
plt.tight_layout()
plt.show()

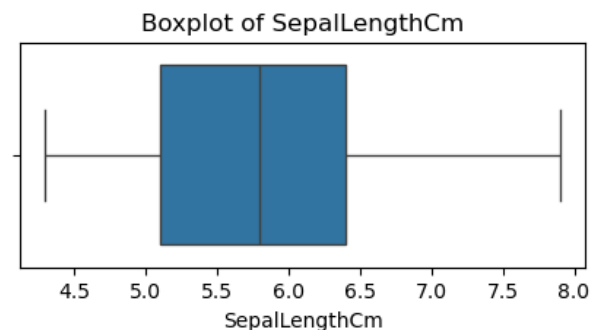
```

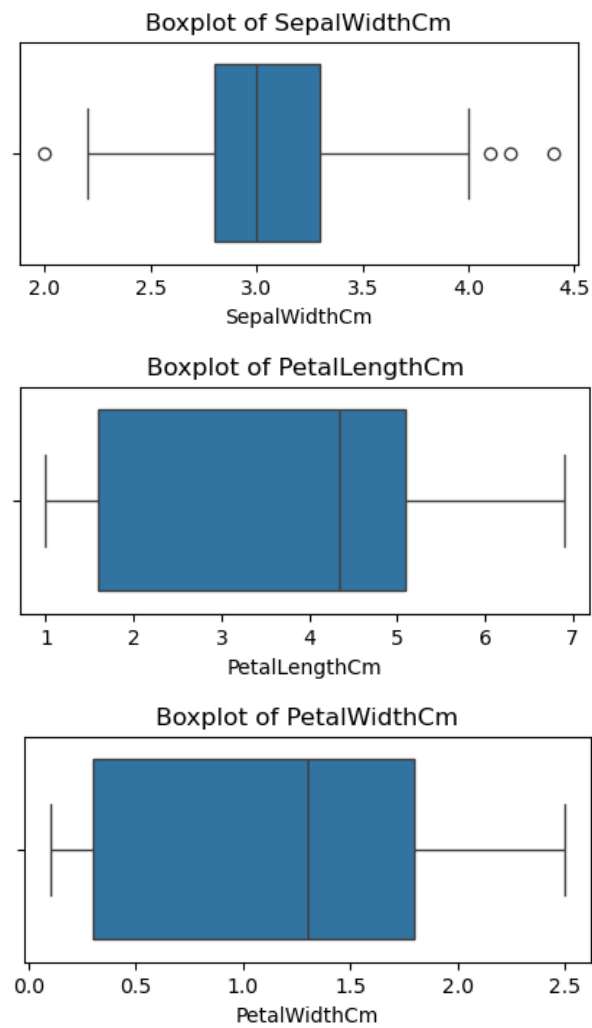


```

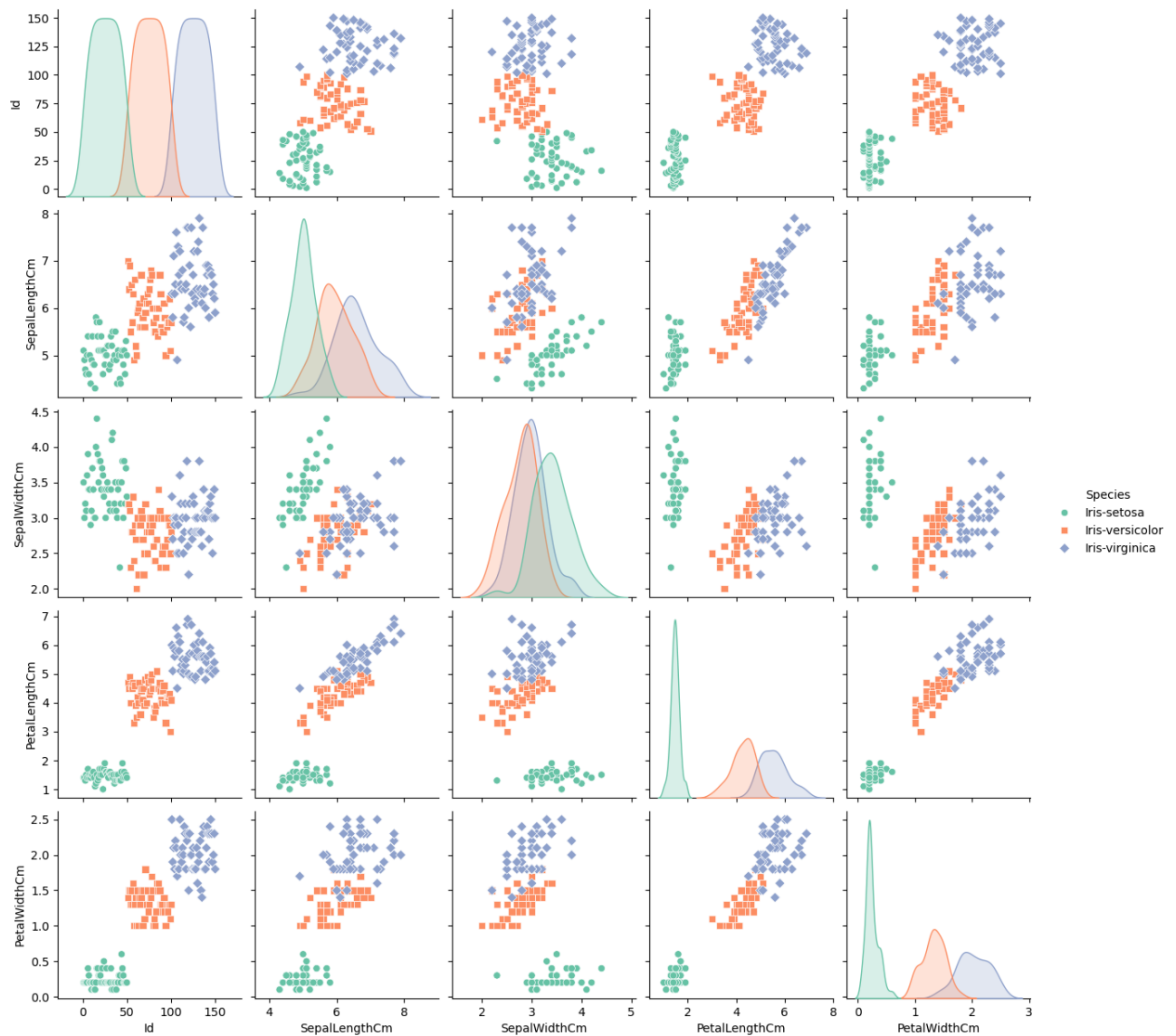
for column in iris_X.columns:
    plt.figure(figsize=(5, 2))
    sns.boxplot(x=iris[column])
    plt.title(f'Boxplot of {column}')
    plt.xlabel(column)
    plt.show()

```





```
sns.pairplot(iris, hue="Species", markers=["o", "s", "D"],  
             palette="Set2")
```

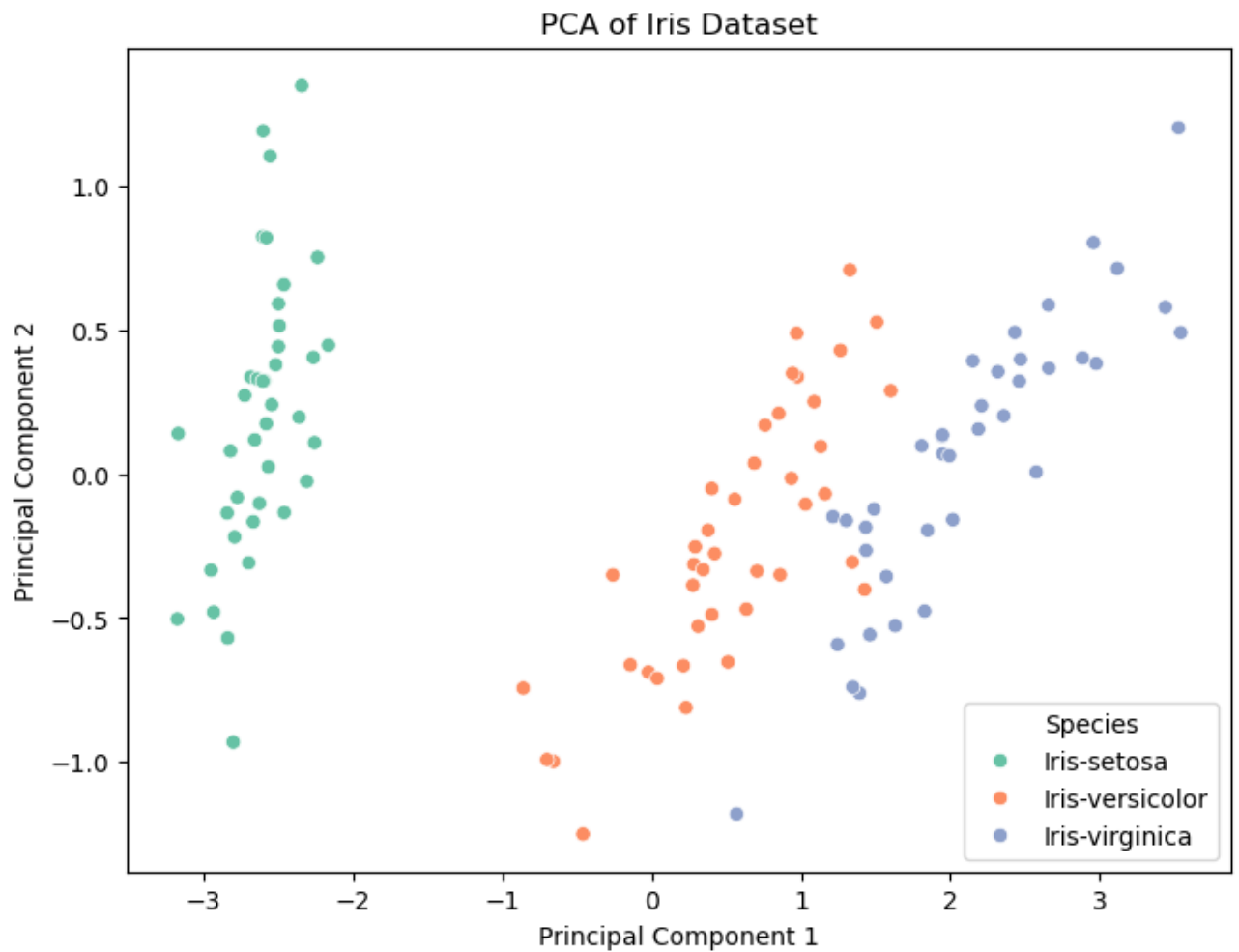


```
X_train, X_test, y_train, y_test = train_test_split(iris_X,
    iris_y, test_size=0.2, random_state=42)
```

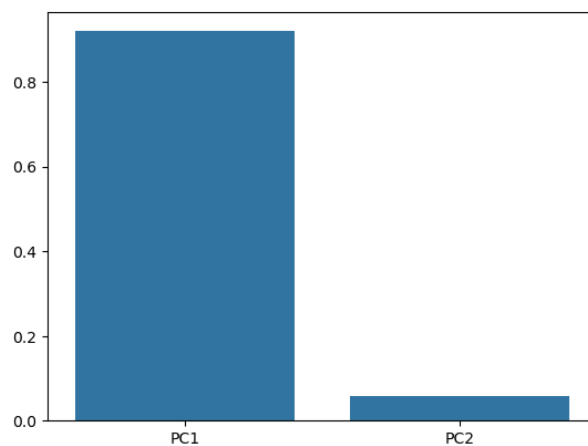
```
pca = PCA(n_components=2)
X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)
```

```
explained_variance = pca.explained_variance_ratio_
print(f"Explained variance by PCA components: {explained_variance
}")
plt.figure(figsize=(8, 6))
sns.scatterplot(x=X_train_pca[:, 0], y=X_train_pca[:, 1], hue=
    y_train, palette="Set2")
plt.title("PCA of Iris Dataset")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
```

```
Explained variance by PCA components: [0.9195717  0.05707944]
```



```
x = [f"PC{i}" for i in range(1, len(pca.explained_variance_ratio_) + 1)]
sns.barplot(x=x, y=explained_variance)
```



```
log_model = LogisticRegression(solver='newton-cholesky')
```

```
log_model.fit(X_train_pca, y_train)
```

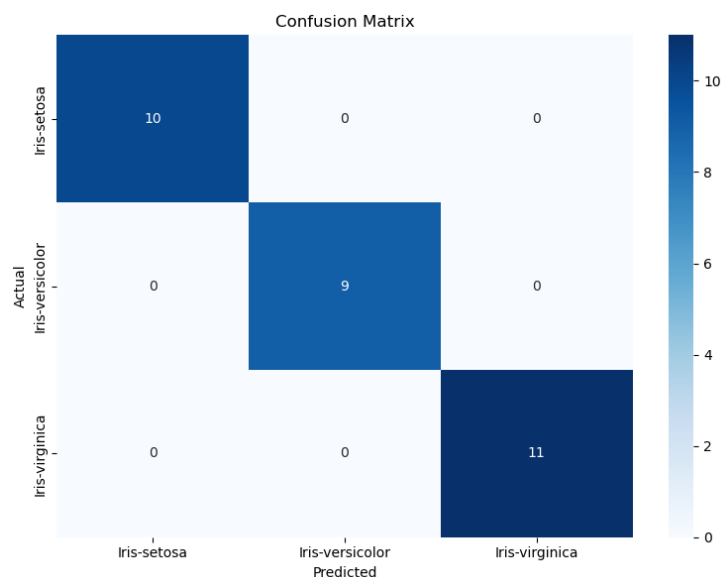
LogisticRegression ⓘ ⓘ
Parameters

```
pred_01 = log_model.predict(X_test_pca)
accuracy = accuracy_score(y_test, pred_01)
print("Accuracy:", accuracy)
conf_matrix = confusion_matrix(y_test, pred_01)
print("Classification Report:\n", classification_report(y_test,
    pred_01))
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
    xticklabels=iris_y.unique(), yticklabels=iris_y.
    unique())
plt.title('Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.tight_layout()
plt.show()
```

Accuracy: 1.0

Classification Report:

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	10
Iris-versicolor	1.00	1.00	1.00	9
Iris-virginica	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30



```
log_model_02 = LogisticRegression(solver='newton-cholesky')
log_model_02.fit(X_train, y_train)
```

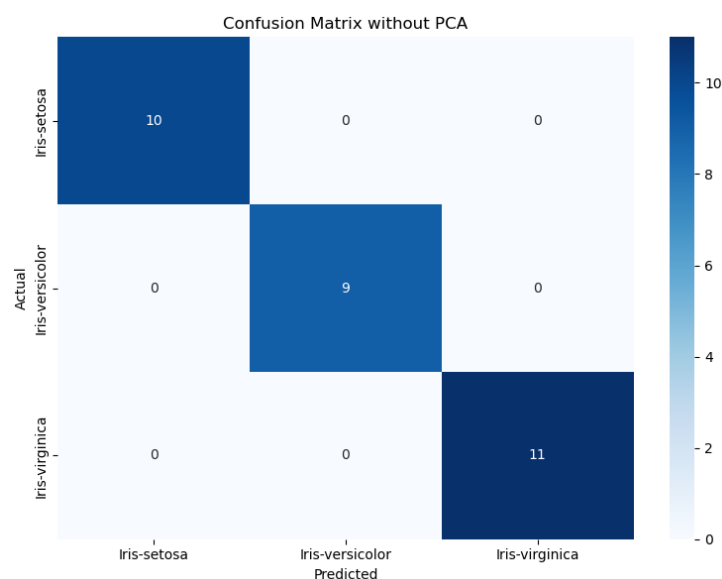
LogisticRegression

Parameters

```
accuracy_02 = accuracy_score(y_test, log_model_02.predict(X_test))
print("Accuracy without PCA:", accuracy_02)
conf_matrix_02 = confusion_matrix(y_test, log_model_02.predict(X_test))
print("Classification Report without PCA:\n",
      classification_report(y_test, log_model_02.predict(X_test)))
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix_02, annot=True, fmt='d', cmap='Blues',
            xticklabels=iris_y.unique(), yticklabels=iris_y.unique())
plt.title('Confusion Matrix without PCA')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.tight_layout()
```

```
Accuracy without PCA: 1.0
Classification Report without PCA:
```

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	10
Iris-versicolor	1.00	1.00	1.00	9
Iris-virginica	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30



AI-Generated

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from mpl_toolkits.mplot3d import Axes3D

# Step 1: Load the Iris dataset
iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = pd.Series(iris.target, name='species')
species_names = iris.target_names

# Combine features and target for easier EDA
df = pd.concat([X, y], axis=1)

# Step 2: EDA
print("Basic Info:")
print(df.info(), "\n")

print("Descriptive Statistics:")
print(df.describe(), "\n")

print("Value Counts:")
print(df['species'].value_counts(), "\n")

# Pairplot
sns.pairplot(df, hue="species", palette='Set2')
plt.suptitle("Pairplot of Iris Features", y=1.02)
plt.show()

# Correlation heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(X.corr(), annot=True, cmap='coolwarm')
plt.title("Feature Correlation Matrix")
plt.show()

# Step 3: Standardize the data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Step 4: Apply PCA
pca = PCA()
X_pca = pca.fit_transform(X_scaled)

# Explained variance
explained_var = pca.explained_variance_ratio_
```

```

cum_var = np.cumsum(explained_var)

# Plot explained variance
plt.figure(figsize=(8, 5))
plt.bar(range(1, len(explained_var)+1), explained_var, alpha=0.7,
        label='Individual')
plt.plot(range(1, len(cum_var)+1), cum_var, marker='o', color='
        red', label='Cumulative')
plt.xticks(range(1, len(explained_var)+1))
plt.ylabel('Explained Variance Ratio')
plt.xlabel('Principal Component')
plt.title('Explained Variance by PCA Components')
plt.legend()
plt.grid(True)
plt.show()

# Step 5: Visualize the transformed data (2D)
pca_2d = PCA(n_components=2)
X_2d = pca_2d.fit_transform(X_scaled)

plt.figure(figsize=(8, 6))
sns.scatterplot(x=X_2d[:, 0], y=X_2d[:, 1], hue=species_names[y],
                palette='Set2')
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.title('PCA: 2D Projection of Iris Dataset')
plt.legend(title='Species')
plt.grid(True)
plt.show()

# Step 6: Optional 3D Visualization
pca_3d = PCA(n_components=3)
X_3d = pca_3d.fit_transform(X_scaled)

fig = plt.figure(figsize=(10, 7))
ax = fig.add_subplot(111, projection='3d')
scatter = ax.scatter(X_3d[:, 0], X_3d[:, 1], X_3d[:, 2], c=y,
                    cmap='Set2', edgecolor='k')
ax.set_title("PCA: 3D Projection of Iris Dataset")
ax.set_xlabel("PC1")
ax.set_ylabel("PC2")
ax.set_zlabel("PC3")
# plt.legend(handles=scatter.legend_elements()[0], labels=
#             species_names, title="Species")
plt.show()

# Step 7: PCA Effectiveness Summary
print("PCA Explained Variance Ratios:")
for i, ratio in enumerate(explained_var):
    print(f"  PC{i+1}: {ratio:.4f}")

```

```
print("\nCumulative Variance (first 2 PCs): {:.2f}%".format(
    cum_var[1] * 100))
```

Comments on comparisons

- The AI generated code has performed EDA and PCA on the iris dataset, fetched using the built-in load_iris function from scikit-learn, while the code I have written uses a CSV file to load the dataset.
- The AI code uses PCA to reduce the dataset to 2D and 3D visualizations, while my code applies PCA to reduce the dataset to 2 components and uses logistic regression for classification and comparison of performance with and without PCA.

Verified By

Cycle II

Experiment 4

Problem

Implement a method to compute the Euclidean distance between two grayscale images.

Aim and Objectives

To understand the concept of Euclidean distance and implement a method to compute it between two grayscale images, demonstrating the application of this metric in image processing.

Program Code

```
import cupy as cp
import cv2
import matplotlib.pyplot as plt

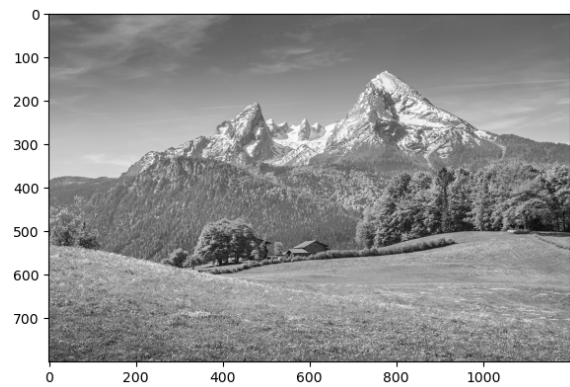
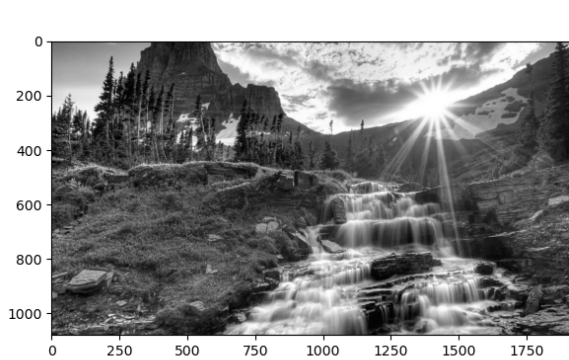
def measure_euclid_dist(img_01, img_02):
    if img_01.shape != img_02.shape:
        img_02 = cv2.resize(img_02, (img_01.shape[1], img_01.
            shape[0]), interpolation=cv2.INTER_LINEAR)
    img_01 = img_01.flatten()
    img_02 = img_02.flatten()
    img_01 = cp.asarray(img_01, dtype=cp.float32)
    img_02 = cp.asarray(img_02, dtype=cp.float32)

    euclid = cp.linalg.norm(img_01 - img_02, axis=-1)

    euclid = cp.asnumpy(euclid)
    return euclid

image_01 = cv2.imread("../images/scenery_01.jpg", cv2.
    IMREAD_GRAYSCALE)
image_02 = cv2.imread("../images/scenery_02.jpg", cv2.
    IMREAD_GRAYSCALE)

plt.subplots(1, 2, figsize=(15, 10))
plt.imshow(image_01, cmap='gray')
plt.subplot(1, 2, 1)
plt.imshow(image_02, cmap='gray')
plt.show()
```



```
image_01.shape
```

```
(800, 1200)
```

```
image_02.shape
```

```
(1080, 1920)
```

```
print("Euclidean distance between images:", measure_euclid_dist(
    image_01, image_02))
```

```
Euclidean distance between images: 82911.17
```

AI-Generated

```
from PIL import Image
import numpy as np
from numpy.linalg import norm

def load_and_preprocess_image(path, size=(256, 256), grayscale=
    True):
    """
    Load and preprocess the image.
    Resize and optionally convert to grayscale.
    """
    img = Image.open(path)
    if grayscale:
        img = img.convert("L") # Convert to grayscale
    img = img.resize(size)     # Resize to uniform size
    return np.array(img)

def euclidean_distance(img1_array, img2_array):
    """
    Compute the Euclidean distance between two image arrays.
    """
    if img1_array.shape != img2_array.shape:
        raise ValueError("Images must be of the same shape")
```

```

# Flatten the images into 1D vectors
vec1 = img1_array.flatten()
vec2 = img2_array.flatten()

# Compute Euclidean distance
distance = norm(vec1 - vec2)
return distance

# Example usage
if __name__ == "__main__":
    img_path1 = "../images/scenery_01.jpg"
    img_path2 = "../images/scenery_02.jpg"

    try:
        img1 = load_and_preprocess_image(img_path1)
        img2 = load_and_preprocess_image(img_path2)

        distance = euclidean_distance(img1, img2)
        print(f"Euclidean distance between the images: {distance :.2f}")
    except Exception as e:
        print("Error:", e)

```

Comments on comparisons

- The code I have written uses GPU acceleration with CuPy and opencv for image processing and distance computation whereas the AI provided code uses CPU-based libraries like NumPy and PIL.
- The AI provided code included explicit image resizing and grayscale conversion for preprocessing.

Verified By

Cycle II

Experiment 5

Problem

Apply Principal Component Analysis (PCA) and Linear Discriminant Analysis (LDA) on MNIST dataset. Compare the performance of PCA vs. LDA for classifying handwritten digits (MNIST dataset).

Aim and Objectives

To understand the differences between PCA and LDA in terms of dimensionality reduction and classification performance on the MNIST dataset, and to implement both techniques to compare their effectiveness in classifying handwritten digits.

Program Code

```
import pandas as pd
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import
    LinearDiscriminantAnalysis
from sklearn.metrics import classification_report,
    confusion_matrix
from sklearn.neighbors import KNeighborsClassifier
import pandas as pd
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
```

```
Loading the MNIST train and test csv
```

```
image_01 = cv2.imread("../images/scenery_01.jpg", cv2.
    IMREAD_GRAYSCALE)
image_02 = cv2.imread("../images/scenery_02.jpg", cv2.
    IMREAD_GRAYSCALE)

plt.subplots(1, 2, figsize=(15, 10))
plt.imshow(image_01, cmap='gray')
plt.subplot(1, 2, 1)
plt.imshow(image_02, cmap='gray')
plt.show()
```

```
train_set = pd.read_csv("../datasets/mnist_train.csv")
test_set = pd.read_csv("../datasets/mnist_test.csv")
```

```
train_set["label"].unique()
```

```
array([5, 0, 4, 1, 9, 2, 3, 6, 7, 8])
```

```
train_set
```

	label	1x1	1x2	1x3	1x4	1x5	1x6	1x7	1x8	1x9	...	28x19	28x20	28x21	28x22	28x23	28x24
0	5	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0
2	4	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0
3	1	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0
4	9	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
59995	8	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0
59996	3	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0
59997	5	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0
59998	6	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0
59999	8	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0

60000 rows × 785 columns

```
test_set
```

	label	1x1	1x2	1x3	1x4	1x5	1x6	1x7	1x8	1x9	...	28x19	28x20	28x21	28x22	28x23	28x24
0	7	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0
1	2	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0
2	1	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0
4	4	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
9995	2	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0
9996	3	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0
9997	4	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0
9998	5	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0
9999	6	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0

10000 rows × 785 columns

```
train_set.describe()
```

	label	1x1	1x2	1x3	1x4	1x5	1x6	1x7	1x8	1x9	...
count	60000.000000	60000.0	60000.0	60000.0	60000.0	60000.0	60000.0	60000.0	60000.0	60000.0	...
mean	4.453933	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...
std	2.889270	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...
min	0.000000	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...
25%	2.000000	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...
50%	4.000000	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...
75%	7.000000	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...
max	9.000000	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...

8 rows × 785 columns

```
test_set.describe()
```


	label	1x1	1x2	1x3	1x4	1x5	1x6	1x7	1x8	1x9	...
count	10000.000000	10000.0	10000.0	10000.0	10000.0	10000.0	10000.0	10000.0	10000.0	10000.0	...
mean	4.443400	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...
std	2.895865	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...
min	0.000000	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...
25%	2.000000	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...
50%	4.000000	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...
75%	7.000000	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...
max	9.000000	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...

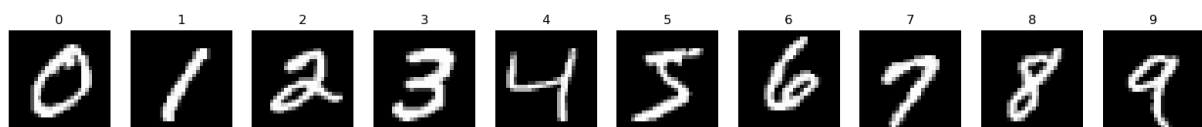
8 rows × 785 columns

Euclidean distance between images: 82911.17

The MNIST dataset contains 10 classes(0 to 9) of handwritten digits each of size 28x28

```
plt.subplots(1, 10, figsize=(20, 17))

for i in range(10):
    plt.subplot(1, 10, i + 1)
    # plt.imshow(train_set[train_set["label"] == i].values
    #           [0][1:].reshape(28, 28), cmap="gray")
    image = train_set[train_set["label"] == i].to_numpy()[0][1:].
        reshape(28, 28)
    plt.imshow(image, cmap="gray")
    plt.axis("off")
    plt.title(i)
plt.show()
```



```
train_set_copy = train_set.copy()
train_set_copy = train_set_copy.drop(columns=["label"])
```

```
test_set_copy = test_set.copy()
test_set_copy = test_set_copy.drop(columns=["label"])
```

Scaling the values of both the sets

```
std_scaler = StandardScaler()
train_set_copy = std_scaler.fit_transform(train_set_copy)
test_set_copy = std_scaler.transform(test_set_copy)
```

Applying PCA on the dataset

```
pca = PCA(n_components=50)
train_set_pca = pca.fit_transform(train_set_copy)
pca.explained_variance_ratio_
```

```
array([0.09704664, 0.07095924, 0.06169089, 0.05389419,
       0.04868797,
       0.04312231, 0.0327193 , 0.02883895, 0.02762029,
       0.02357001,
       0.0210919 , 0.02022991, 0.01715818, 0.01692111,
       0.01578641,
       0.01482953, 0.01324561, 0.01276897, 0.01187263,
       0.01152684,
       0.01066166, 0.01006713, 0.00953573, 0.00912544,
       0.00883405,
       0.00839319, 0.00812579, 0.00786366, 0.00744733,
       0.00690859,
       0.00658094, 0.00648148, 0.00602615, 0.00586582,
       0.00570021,
       0.00543628, 0.00505786, 0.00487859, 0.00481429,
       0.00472266,
       0.00456747, 0.00444836, 0.00418501, 0.00398215,
       0.00384975,
       0.00375103, 0.00362009, 0.00351591, 0.00340058,
       0.00321874])
```

```
test_set_pca = pca.fit_transform(test_set_copy)
```

Applying LDA on the dataset

```
lda = LinearDiscriminantAnalysis(n_components=9)
train_set_lda = lda.fit_transform(train_set_copy, train_set["
    label"])
lda.explained_variance_ratio_
```

```
array([0.2392286 , 0.20180995, 0.17849695, 0.10652571,
       0.09406712,
       0.06906025, 0.04973746, 0.03429077, 0.0267832 ])
```

```
test_set_lda = lda.fit_transform(test_set_copy, test_set["label"
    ])
```

Training two KNN models on the two train sets

```
knn_01 = KNeighborsClassifier(n_neighbors=5)
knn_02 = KNeighborsClassifier(n_neighbors=5)
```

```
knn_01.fit(train_set_pca, train_set["label"])
```

▼ KNeighborsClassifier ⓘ
KNeighborsClassifier()

```
knn_02.fit(train_set_lda, train_set["label"])
```

▼ KNeighborsClassifier ⓘ
KNeighborsClassifier()

Evaluating the performance of the two SVMs

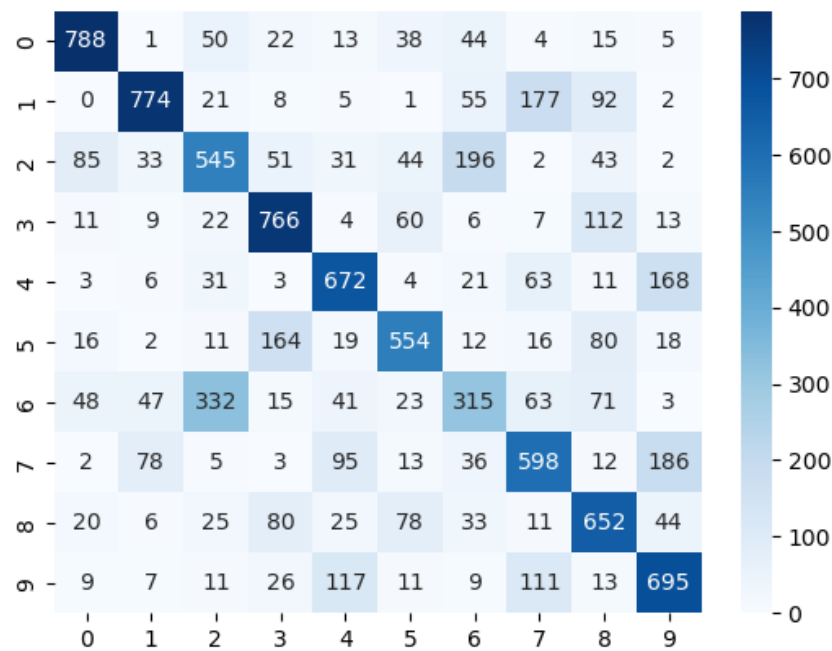
```
pred_01 = knn_01.predict(test_set_pca)
pred_02 = knn_02.predict(test_set_lda)
```

```
print("PCA Classification Report")
print(classification_report(test_set["label"], pred_01))
```

```
PCA Classification Report
```

	precision	recall	f1-score	support
0	0.80	0.80	0.80	980
1	0.80	0.68	0.74	1135
2	0.52	0.53	0.52	1032
3	0.67	0.76	0.71	1010
4	0.66	0.68	0.67	982
5	0.67	0.62	0.64	892
6	0.43	0.33	0.37	958
7	0.57	0.58	0.57	1028
8	0.59	0.67	0.63	974
9	0.61	0.69	0.65	1009
accuracy			0.64	10000
macro avg	0.63	0.63	0.63	10000
weighted avg	0.64	0.64	0.63	10000

```
confusion_matrix_01 = confusion_matrix(test_set["label"], pred_01)
sns.heatmap(confusion_matrix_01, annot=True, fmt="d", cmap="Blues")
```

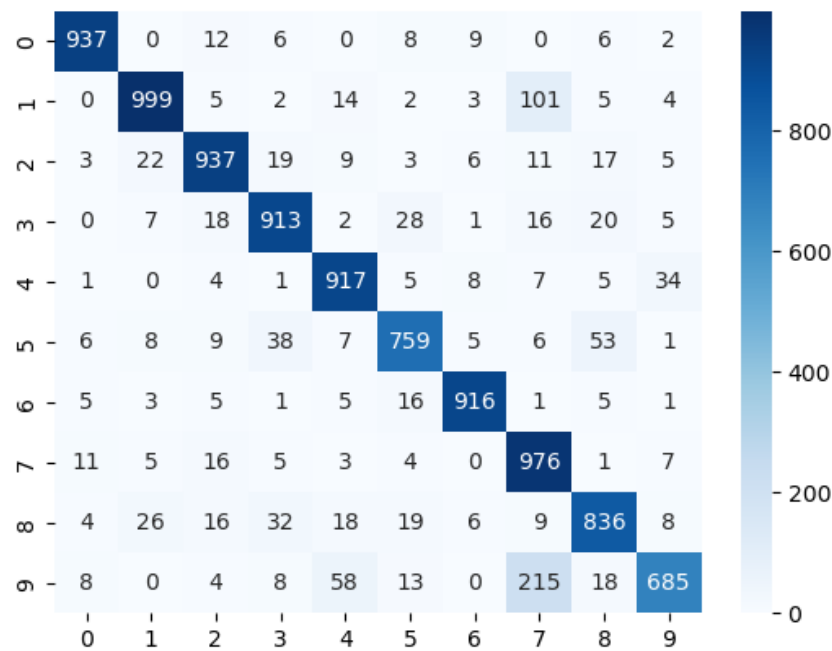


```
print("LDA Classification Report")
print(classification_report(test_set["label"], pred_02))
```

```
LDA Classification Report
```

	precision	recall	f1-score	support
0	0.96	0.96	0.96	980
1	0.93	0.88	0.91	1135
2	0.91	0.91	0.91	1032
3	0.89	0.90	0.90	1010
4	0.89	0.93	0.91	982
5	0.89	0.85	0.87	892
6	0.96	0.96	0.96	958
7	0.73	0.95	0.82	1028
8	0.87	0.86	0.86	974
9	0.91	0.68	0.78	1009
accuracy			0.89	10000
macro avg	0.89	0.89	0.89	10000
weighted avg	0.89	0.89	0.89	10000

```
confusion_matrix_02 = confusion_matrix(test_set["label"], pred_02)
sns.heatmap(confusion_matrix_02, annot=True, fmt="d", cmap="Blues")
```



AI-Generated

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_openml
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import
    LinearDiscriminantAnalysis as LDA
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report

# Load MNIST dataset
print("Loading MNIST dataset...")
X, y = fetch_openml('mnist_784', version=1, return_X_y=True,
    as_frame=False)
y = y.astype(int)

# Train-test split
print("Splitting dataset...")
X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.2, random_state=42)

# Apply PCA (to 50 components)
print("Applying PCA...")
pca = PCA(n_components=50)
X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)

# Apply LDA (for classification - full 9 components)
print("Applying LDA for classification...")
lda_full = LDA(n_components=9)
```

```

X_train_lda = lda_full.fit_transform(X_train, y_train)
X_test_lda = lda_full.transform(X_test)

# Train KNN on PCA
print("Training KNN on PCA...")
knn_pca = KNeighborsClassifier(n_neighbors=3)
knn_pca.fit(X_train_pca, y_train)
y_pred_pca = knn_pca.predict(X_test_pca)
acc_pca = accuracy_score(y_test, y_pred_pca)

# Train KNN on LDA
print("Training KNN on LDA...")
knn_lda = KNeighborsClassifier(n_neighbors=3)
knn_lda.fit(X_train_lda, y_train)
y_pred_lda = knn_lda.predict(X_test_lda)
acc_lda = accuracy_score(y_test, y_pred_lda)

# Print accuracy
print("\n--- Results ---")
print(f"PCA Accuracy: {acc_pca:.4f}")
print(f"LDA Accuracy: {acc_lda:.4f}")

# Classification Reports
print("\nClassification Report (PCA):")
print(classification_report(y_test, y_pred_pca))
print("\nClassification Report (LDA):")
print(classification_report(y_test, y_pred_lda))

# PCA Explained Variance Plot
plt.figure(figsize=(8, 4))
plt.plot(np.cumsum(pca.explained_variance_ratio_[:100]), marker='o')
plt.title("PCA - Cumulative Explained Variance")
plt.xlabel("Number of Components")
plt.ylabel("Cumulative Variance Explained")
plt.grid(True)
plt.tight_layout()
plt.show()

# LDA Visualization (2D)
print("Visualizing LDA in 2D...")
lda_2d = LDA(n_components=2)
X_lda_2d = lda_2d.fit_transform(X_train, y_train)

plt.figure(figsize=(10, 8))
scatter = plt.scatter(X_lda_2d[:, 0], X_lda_2d[:, 1], c=y_train,
                      cmap='tab10', alpha=0.7, s=10)
plt.title("LDA 2D Visualization of MNIST Training Set")
plt.xlabel("LDA Component 1")
plt.ylabel("LDA Component 2")
plt.legend(*scatter.legend_elements(), title="Digits", loc="best")

```

```
)  
plt.grid(True)  
plt.tight_layout()  
plt.show()
```

Comments on comparisons

- The AI generated code has performed PCA and LDA on the MNIST dataset, fetched using the built-in `fetch_openml` function from `scikit-learn`, while the code I have written uses a CSV file to load the dataset.
- Both codes apply PCA to reduce the dataset to 50 components and LDA to reduce the dataset to 9 components, but the AI code uses KNN with 3 neighbors for classification while my code uses KNN with 5 neighbors.
- The AI code includes a visualization of the cumulative explained variance for PCA and a 2D visualization of LDA components, while my code focuses on classification performance and confusion matrices.

Verified By

Cycle II

Experiment 6

Problem

Load an image dataset (eg: Iris dataset). Compute the following distances between sample points.

- Hamming Distance
- Euclidean Distance
- Manhattan (City Block) Distance

Aim and Objectives

To understand and implement different distance metrics (Hamming, Euclidean, and Manhattan) on a dataset, specifically the Iris dataset, to analyze the relationships between sample points. Program Code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os
from scipy.spatial.distance import euclidean, hamming, cityblock
```

```
setosa = os.listdir("../datasets/iris/iris-setosa/")
versicolor = os.listdir("../datasets/iris/iris-versicolour/")
virginica = os.listdir("../datasets/iris/iris-virginica/")
```

```
x = np.random.randint(0, len(setosa), 1)
image = cv2.imread("../datasets/iris/iris-setosa/" + setosa[x
    [0]], cv2.IMREAD_GRAYSCALE)

y = np.random.randint(0, len(versicolor), 1)
image2 = cv2.imread("../datasets/iris/iris-versicolour/" +
    versicolor[y[0]], cv2.IMREAD_GRAYSCALE)

z = np.random.randint(0, len(virginica), 1)
image3 = cv2.imread("../datasets/iris/iris-virginica/" +
    virginica[z[0]], cv2.IMREAD_GRAYSCALE)

plt.figure(figsize=(10, 10))
plt.subplot(1,3,1)
plt.imshow(image, cmap='gray')
plt.title("Iris Setosa")
plt.axis('off')
plt.subplot(1,3,2)
plt.imshow(image2, cmap='gray')
plt.title("Iris Versicolor")
```



```
plt.axis('off')
plt.subplot(1,3,3)
plt.imshow(image3, cmap='gray')
plt.title("Iris Virginica")
plt.axis('off')
plt.tight_layout()
plt.show()
```



Calculating the hamming distance between two images

```
image_bin = cv2.adaptiveThreshold(image, 255, cv2.
    ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 11, 2)
image2_bin = cv2.adaptiveThreshold(image2, 255, cv2.
    ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 11, 2)

image3_bin = cv2.adaptiveThreshold(image3, 255, cv2.
    ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 11, 2)
```

```
print("Hamming Distance between Setosa and Versicolor: ", ham_1)
print("Hamming Distance between Setosa and Virginica: ", ham_2)
print("Hamming Distance between Versicolor and Virginica: ",
    ham_3)
```

```
Hamming Distance between Setosa and Versicolor:
    0.385467529296875
Hamming Distance between Setosa and Virginica:  0.374298095703125
Hamming Distance between Versicolor and Virginica:
    0.307220458984375
```

```
euc_1 = euclidean(image_bin.flatten(), image2_bin.flatten())
euc_2 = euclidean(image_bin.flatten(), image3_bin.flatten())
euc_3 = euclidean(image2_bin.flatten(), image3_bin.flatten())

print("Euclidean Distance between Setosa and Versicolor: ", euc_1
    )
print("Euclidean Distance between Setosa and Virginica: ", euc_2)
print("Euclidean Distance between Versicolor and Virginica: ",
    euc_3)
```

```
Euclidean Distance between Setosa and Versicolor:
24284.314237795556
Euclidean Distance between Setosa and Virginica:
23709.85048455599
Euclidean Distance between Versicolor and Virginica:
25511.670858648205
```

```
cityblock_1 = cityblock(image_bin.flatten(), image2_bin.flatten()
)
cityblock_2 = cityblock(image_bin.flatten(), image3_bin.flatten()
)
cityblock_3 = cityblock(image2_bin.flatten(), image3_bin.flatten
())

print("Cityblock Distance between Setosa and Versicolor: ",
      cityblock_1)
print("Cityblock Distance between Setosa and Virginica: ",
      cityblock_2)
print("Cityblock Distance between Versicolor and Virginica: ",
      cityblock_3)
```

```
Cityblock Distance between Setosa and Versicolor: 2328788
Cityblock Distance between Setosa and Virginica: 2220360
Cityblock Distance between Versicolor and Virginica: 2562420
```

AI-Generated

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from scipy.spatial.distance import euclidean, cityblock, hamming
from itertools import combinations

# Load sample image dataset (use this or replace with your own
image loader)
digits = load_digits()
images = digits.images
labels = digits.target

# Flatten images into 1D vectors
flattened_images = images.reshape((images.shape[0], -1))

# Select a few samples for demonstration
sample_indices = [0, 1, 2] # You can change these
samples = flattened_images[sample_indices]
sample_labels = labels[sample_indices]

# Compute and display distances between pairs
print("Distance Metrics Between Sample Images:\n")

for (i, a), (j, b) in combinations(enumerate(samples), 2):
```

```

eu_dist = euclidean(a, b)
man_dist = cityblock(a, b)
ham_dist = hamming(a > 0, b > 0)  # Convert to binary for
    Hamming

print(f"Image {sample_labels[i]} vs Image {sample_labels[j]}:
    ")
print(f"    Euclidean Distance : {eu_dist:.4f}")
print(f"    Manhattan Distance : {man_dist:.4f}")
print(f"    Hamming Distance    : {ham_dist:.4f}\n")

# Optional: Visualize the sample images
fig, axes = plt.subplots(1, len(sample_indices), figsize=(10, 3))
for ax, idx in zip(axes, sample_indices):
    ax.imshow(images[idx], cmap='gray')
    ax.set_title(f"Label: {labels[idx]}")
    ax.axis("off")
plt.tight_layout()
plt.show()

```

Comments on comparisons

- The AI-generated code uses the ‘load_digits’ function from scikit-learn to load a sample image dataset, while my code uses images from the Iris dataset.
- The AI code computes distances between pairs of images using Euclidean, Manhattan, and Hamming metrics, similar to my code.
- The AI code uses ‘combinations’ from the ‘itertools’ module to compute distances between all pairs of selected samples, while my code computes distances between three specific images from the Iris dataset.

Verified By

Cycle II

Experiment 7

Problem

Perform union, intersection, and complement operations on two fuzzy sets. Visualize the fuzzy sets and their operations. Implement De Morgans law on Fuzzy sets.

Aim and Objectives

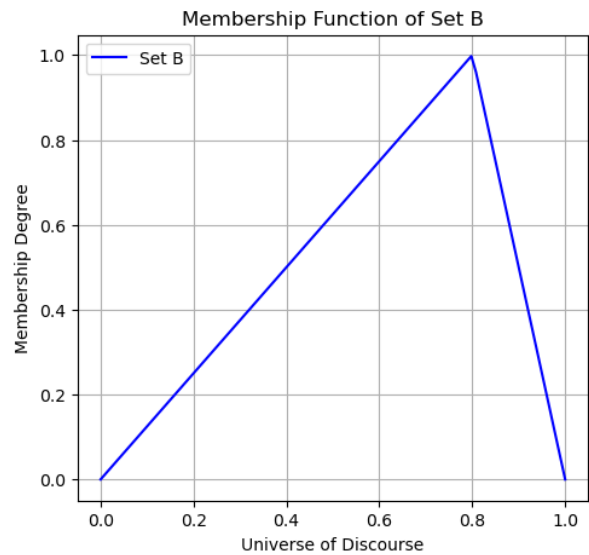
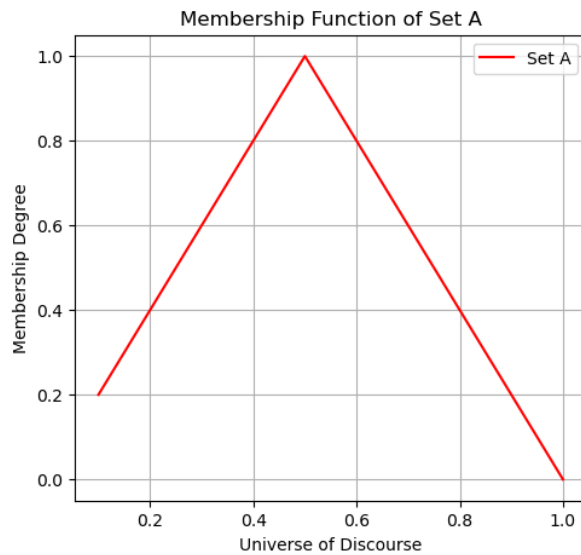
To understand and implement fuzzy set operations such as union, intersection, and complement, and to visualize these operations. Additionally, to apply De Morgan's law on fuzzy sets. Program Code

```
import matplotlib.pyplot as plt
import random
import numpy as np
import skfuzzy as fuzzy
```

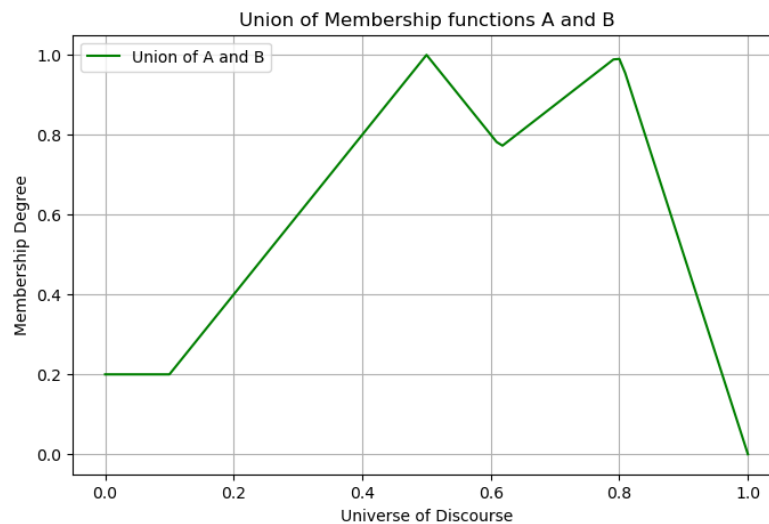
```
set_a = np.linspace(0.1, 1, 100)
set_b = np.linspace(0, 1, 100)
```

```
m_a = fuzzy.membership.trimf(set_a, [0, 0.5, 1])
m_b = fuzzy.membership.trimf(set_b, [0, 0.8, 1])
```

```
plt.subplots(1, 2, figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(set_a, m_a, 'r', label='Set A')
plt.title('Membership Function of Set A')
plt.xlabel('Universe of Discourse')
plt.ylabel('Membership Degree')
plt.legend()
plt.grid()
plt.subplot(1, 2, 2)
plt.plot(set_b, m_b, 'b', label='Set B')
plt.title('Membership Function of Set B')
plt.xlabel('Universe of Discourse')
plt.ylabel('Membership Degree')
plt.legend()
plt.grid()
```

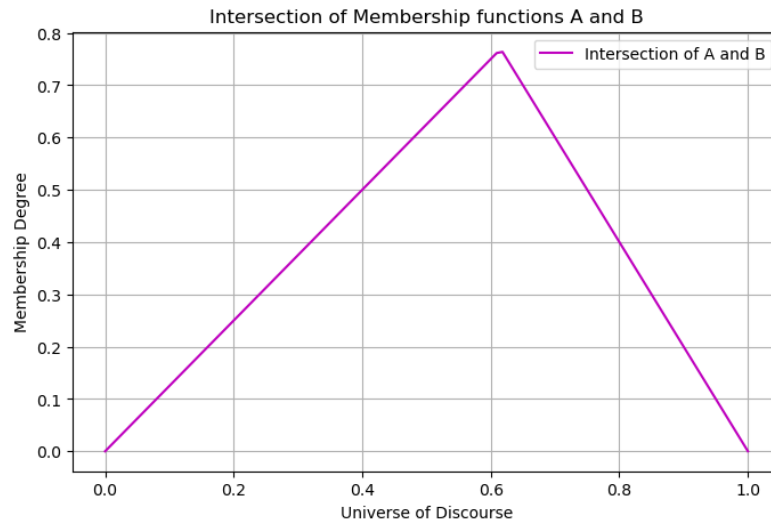


```
union_x, union_y = fuzzy.fuzzy_or(set_a, m_a, set_b, m_b)
plt.figure(figsize=(8,5))
plt.plot(union_x, union_y, 'g', label='Union of A and B')
plt.title('Union of Membership functions A and B')
plt.xlabel('Universe of Discourse')
plt.ylabel('Membership Degree')
plt.legend()
plt.grid()
plt.show()
```

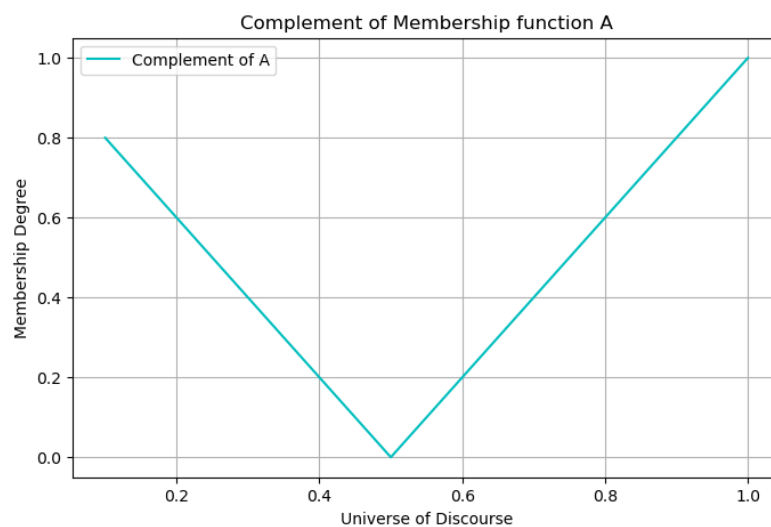


```
intersection_x, intersection_y = fuzzy.fuzzy_and(set_a, m_a,
    set_b, m_b)
plt.figure(figsize=(8,5))
plt.plot(intersection_x, intersection_y, 'm', label='Intersection
    of A and B')
plt.title('Intersection of Membership functions A and B')
plt.xlabel('Universe of Discourse')
```

```
plt.ylabel('Membership Degree')
plt.legend()
plt.grid()
plt.show()
```

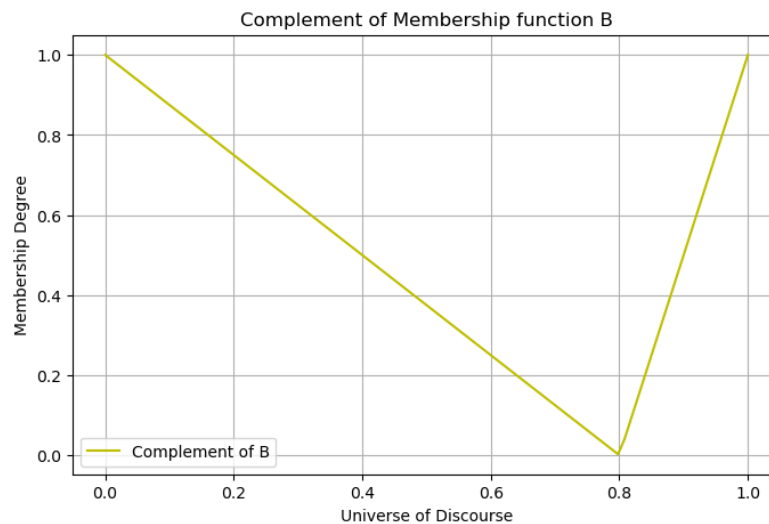


```
complement_x= fuzzy.fuzzy_not(m_a)
plt.figure(figsize=(8,5))
plt.plot(set_a, complement_x, 'c', label='Complement of A')
plt.title('Complement of Membership function A')
plt.xlabel('Universe of Discourse')
plt.ylabel('Membership Degree')
plt.legend()
plt.grid()
plt.show()
```



```
complement_y = fuzzy.fuzzy_not(m_b)
plt.figure(figsize=(8,5))
```

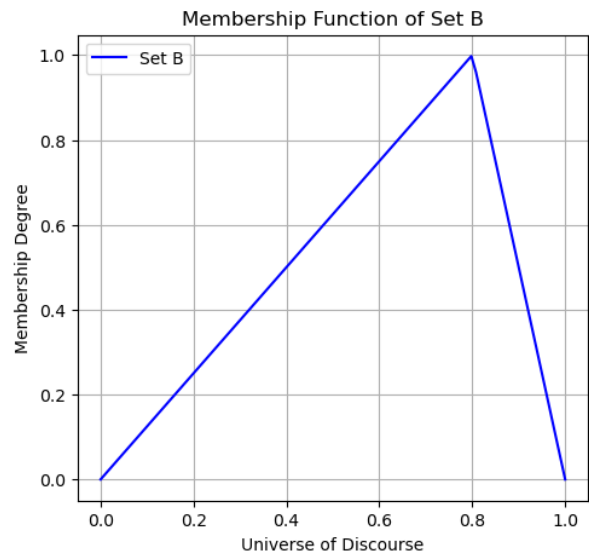
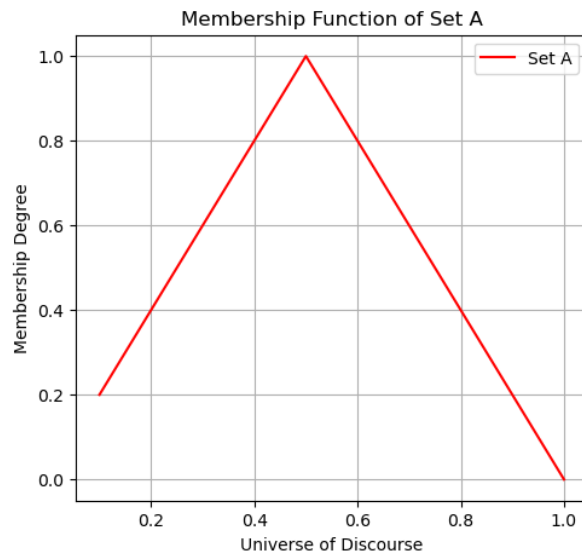
```
plt.plot(set_b, complement_y, 'y', label='Complement of B')
plt.title('Complement of Membership function B')
plt.xlabel('Universe of Discourse')
plt.ylabel('Membership Degree')
plt.legend()
plt.grid()
plt.show()
```



Verifying De Morgan's law using the two fuzzy sets

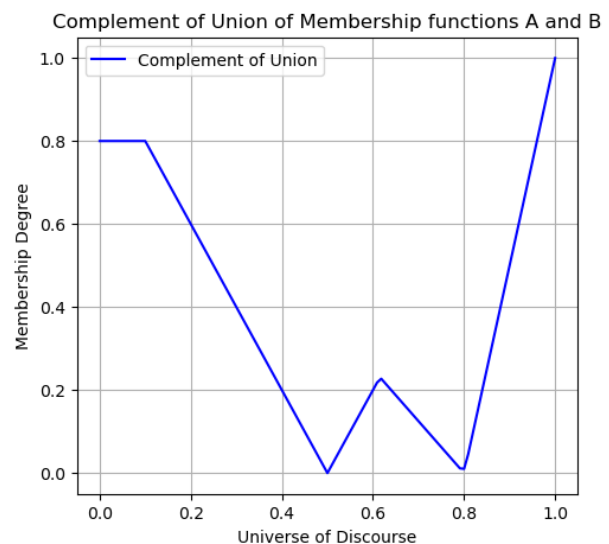
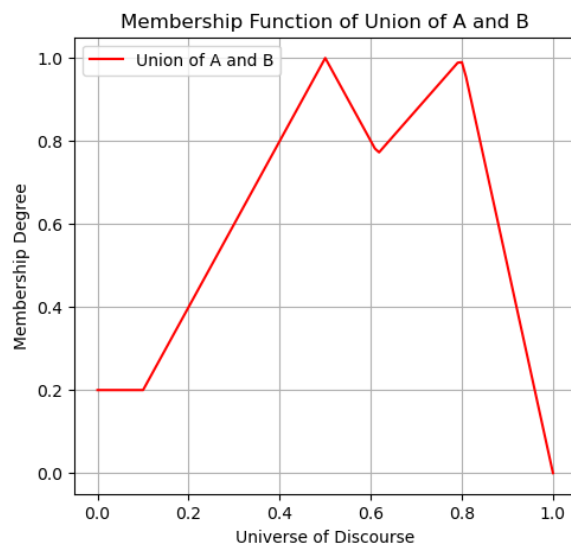
De Morgan's first law states that: $(A \cup B)' = A' \cap B'$

```
plt.subplots(1, 2, figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(set_a, m_a, 'r', label='Set A')
plt.title('Membership Function of Set A')
plt.xlabel('Universe of Discourse')
plt.ylabel('Membership Degree')
plt.legend()
plt.grid()
plt.subplot(1, 2, 2)
plt.plot(set_b, m_b, 'b', label='Set B')
plt.title('Membership Function of Set B')
plt.xlabel('Universe of Discourse')
plt.ylabel('Membership Degree')
plt.legend()
plt.grid()
```



```
union_x, union_y = fuzzy.fuzzy_or(set_a, m_a, set_b, m_b)
complement_x = fuzzy.fuzzy_not(union_y)

plt.subplots(1, 2, figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(union_x, union_y, 'r', label='Union of A and B')
plt.title('Membership Function of Union of A and B')
plt.xlabel('Universe of Discourse')
plt.ylabel('Membership Degree')
plt.legend()
plt.grid()
plt.subplot(1, 2, 2)
plt.plot(union_x, complement_x, 'b', label='Complement of Union')
plt.title('Complement of Union of Membership functions A and B')
plt.xlabel('Universe of Discourse')
plt.ylabel('Membership Degree')
plt.legend()
plt.grid()
```




```

complement_x = fuzzy.fuzzy_not(m_a)
complement_y = fuzzy.fuzzy_not(m_b)

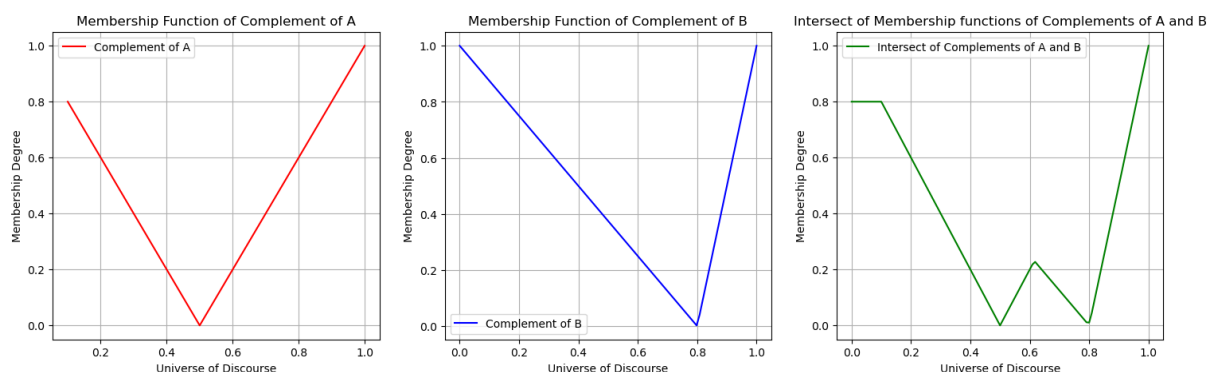
union_x, union_y = fuzzy.fuzzy_and(set_a, complement_x, set_b,
    complement_y)

plt.subplots(1, 3, figsize=(18, 5))
plt.subplot(1, 3, 1)
plt.plot(set_a, complement_x, 'r', label='Complement of A')
plt.title('Membership Function of Complement of A')
plt.xlabel('Universe of Discourse')
plt.ylabel('Membership Degree')
plt.legend()
plt.grid()

plt.subplot(1, 3, 2)
plt.plot(set_b, complement_y, 'b', label='Complement of B')
plt.title('Membership Function of Complement of B')
plt.xlabel('Universe of Discourse')
plt.ylabel('Membership Degree')
plt.legend()
plt.grid()

plt.subplot(1, 3, 3)
plt.plot(union_x, union_y, 'g', label='Intersect of Complements
    of A and B')
plt.title('Intersect of Membership functions of Complements of A
    and B')
plt.xlabel('Universe of Discourse')
plt.ylabel('Membership Degree')
plt.legend()
plt.grid()
plt.show()

```



De Morgan's second law states that: $(A \cap B)' = A' \cup B'$

```

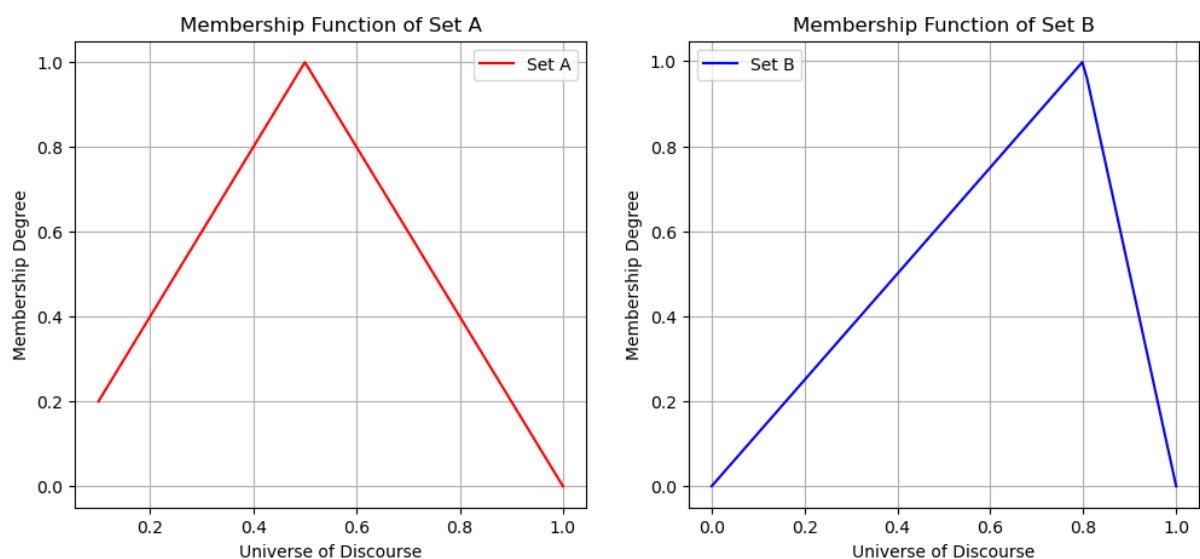
plt.subplots(1, 2, figsize=(12, 5))
plt.subplot(1, 2, 1)

```

```

plt.plot(set_a, m_a, 'r', label='Set A')
plt.title('Membership Function of Set A')
plt.xlabel('Universe of Discourse')
plt.ylabel('Membership Degree')
plt.legend()
plt.grid()
plt.subplot(1, 2, 2)
plt.plot(set_b, m_b, 'b', label='Set B')
plt.title('Membership Function of Set B')
plt.xlabel('Universe of Discourse')
plt.ylabel('Membership Degree')
plt.legend()
plt.grid()

```



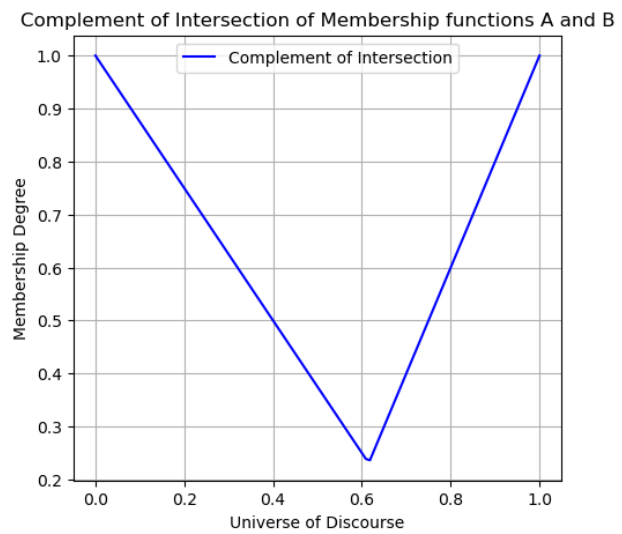
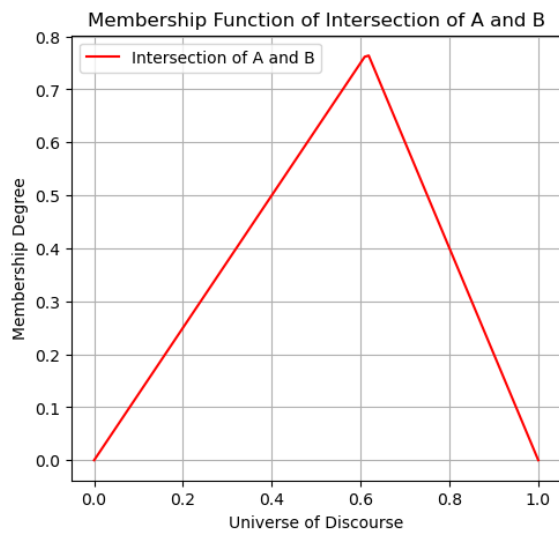
```

intersection_x, intersection_y = fuzzy.fuzzy_and(set_a, m_a,
    set_b, m_b)
complement_x = fuzzy.fuzzy_not(intersection_y)

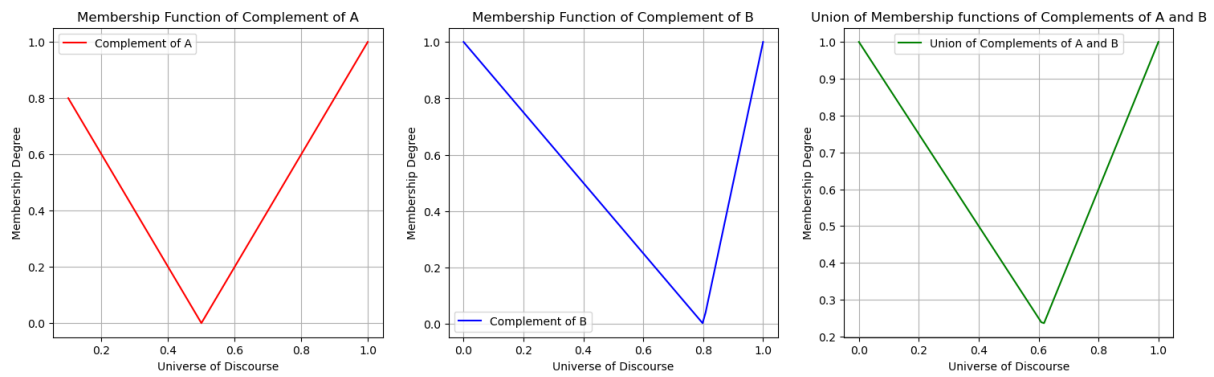
plt.subplots(1, 2, figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(intersection_x, intersection_y, 'r', label='Intersection
    of A and B')
plt.title('Membership Function of Intersection of A and B')
plt.xlabel('Universe of Discourse')
plt.ylabel('Membership Degree')
plt.legend()
plt.grid()
plt.subplot(1, 2, 2)
plt.plot(union_x, complement_x, 'b', label='Complement of
    Intersection')
plt.title('Complement of Intersection of Membership functions A
    and B')
plt.xlabel('Universe of Discourse')
plt.ylabel('Membership Degree')

```

```
plt.legend()
plt.grid()
```



```
complement_x = fuzzy.fuzzy_not(m_a)
complement_y = fuzzy.fuzzy_not(m_b)
union_x, union_y = fuzzy.fuzzy_or(set_a, complement_x, set_b,
                                   complement_y)
plt.subplots(1, 3, figsize=(18, 5))
plt.subplot(1, 3, 1)
plt.plot(set_a, complement_x, 'r', label='Complement of A')
plt.title('Membership Function of Complement of A')
plt.xlabel('Universe of Discourse')
plt.ylabel('Membership Degree')
plt.legend()
plt.grid()
plt.subplot(1, 3, 2)
plt.plot(set_b, complement_y, 'b', label='Complement of B')
plt.title('Membership Function of Complement of B')
plt.xlabel('Universe of Discourse')
plt.ylabel('Membership Degree')
plt.legend()
plt.grid()
plt.subplot(1, 3, 3)
plt.plot(union_x, union_y, 'g', label='Union of Complements of A
    and B')
plt.title('Union of Membership functions of Complements of A and
    B')
plt.xlabel('Universe of Discourse')
plt.ylabel('Membership Degree')
plt.legend()
plt.grid()
plt.show()
```



AI-Generated

```
import numpy as np
import matplotlib.pyplot as plt

# Define the universe of discourse
x = np.linspace(0, 10, 100)

# Define two fuzzy sets A and B (membership functions)
A = np.maximum(0, 1 - abs(x - 3)/2) # Triangle centered at 3
B = np.maximum(0, 1 - abs(x - 7)/2) # Triangle centered at 7

# Fuzzy set operations
union = np.maximum(A, B)
intersection = np.minimum(A, B)
complement_A = 1 - A
complement_B = 1 - B

# De Morgan's Laws
de_morgan_1 = 1 - union
de_morgan_1_rhs = np.minimum(complement_A, complement_B)

de_morgan_2 = 1 - intersection
de_morgan_2_rhs = np.maximum(complement_A, complement_B)

# --- Visualization ---

fig, axs = plt.subplots(3, 2, figsize=(12, 10))
fig.suptitle("Fuzzy Set Operations and De Morgan's Laws",
             fontsize=16)

# Plot original sets
axs[0, 0].plot(x, A, label='A', color='blue')
axs[0, 0].plot(x, B, label='B', color='red')
axs[0, 0].set_title("Original Fuzzy Sets")
axs[0, 0].legend()
axs[0, 0].grid()

# Union and Intersection
axs[0, 1].plot(x, union, label='A union B', color='purple')
```

```

axs[0, 1].plot(x, intersection, label='A intersection B', color='
    green')
axs[0, 1].set_title("Union and Intersection")
axs[0, 1].legend()
axs[0, 1].grid()

# Complements
axs[1, 0].plot(x, complement_A, label="A'", color='blue',
    linestyle='--')
axs[1, 0].plot(x, complement_B, label="B'", color='red',
    linestyle='--')
axs[1, 0].set_title("Complements of A and B")
axs[1, 0].legend()
axs[1, 0].grid()

# De Morgan's Law 1
axs[1, 1].plot(x, de_morgan_1, label="(A union B}'", color='
    orange')
axs[1, 1].plot(x, de_morgan_1_rhs, label="A' intersection B'",
    color='green', linestyle='--')
axs[1, 1].set_title("De Morgan's Law 1")
axs[1, 1].legend()
axs[1, 1].grid()

# De Morgan's Law 2
axs[2, 0].plot(x, de_morgan_2, label="(A intersection B}'", color
    ='orange')
axs[2, 0].plot(x, de_morgan_2_rhs, label="A' union B'", color='
    purple', linestyle='--')
axs[2, 0].set_title("De Morgan's Law 2")
axs[2, 0].legend()
axs[2, 0].grid()

# Hide unused subplot
axs[2, 1].axis('off')

plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()

```

Comments on comparisons

- The AI-generated code uses NumPy and Matplotlib to define and visualize fuzzy sets, while my code uses the skfuzzy library for fuzzy operations.
- The AI code visualizes the fuzzy sets and their operations in a single figure with multiple subplots, while my code visualizes each operation in separate figures.

Verified By

Cycle II

Experiment 8

Problem

Write a Python program to plot Triangular, Trapezoidal, and Bell-shaped membership functions using NumPy and Matplotlib

Aim and Objectives

To understand the different types of membership functions in fuzzy logic, specifically Triangular, Trapezoidal, and Bell-shaped membership functions, and to visualize them using NumPy and Matplotlib. Program Code

```
import numpy as np
import matplotlib.pyplot as plt
```

```
set_a = np.linspace(0, 10, 5)
```

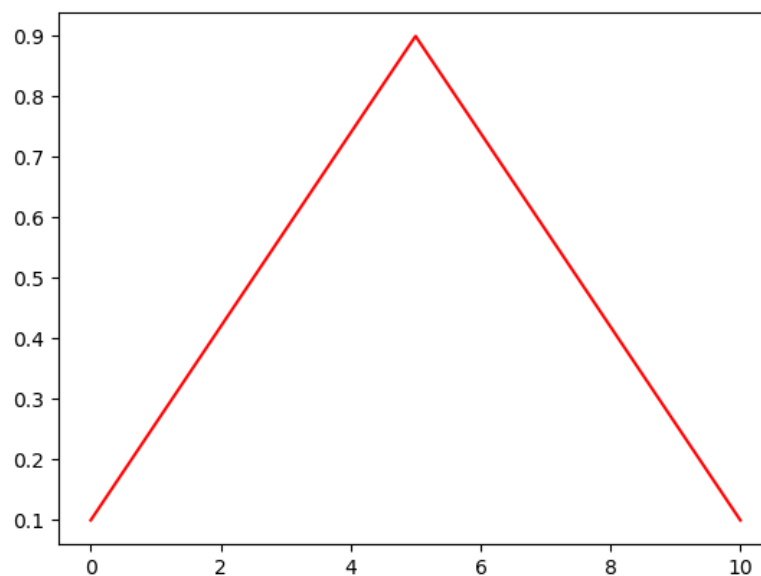
```
set_a
```

```
array([ 0. ,  2.5,  5. ,  7.5, 10. ])
```

```
Triangular Membership Function
```

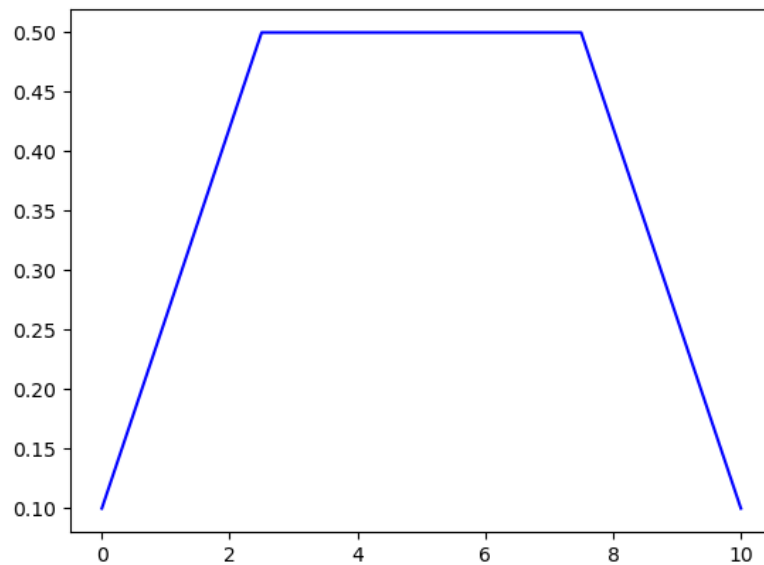
```
Triangular Membership Function
```

```
y = [0.1, 0.5, 0.9, 0.5, 0.1]
plt.plot(set_a, y, 'r', label = 'Fuzzy Set A')
```



```
Trapezoidal Membership Function
```

```
y = [0.1, 0.5, 0.5, 0.5, 0.1]
plt.plot(set_a, y, 'b', label = 'Fuzzy Set B')
```



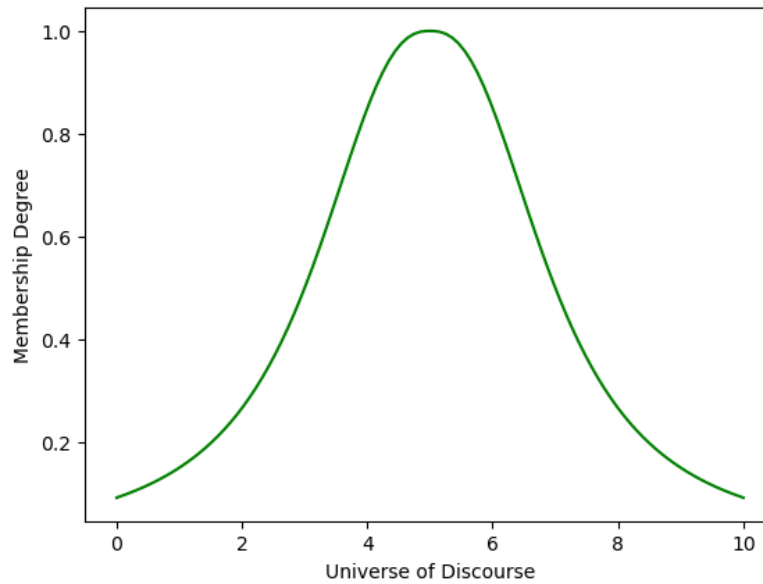
Bell-shaped Membership Function

```
set_a = np.linspace(0, 10, 100)

width = 2
slope = 2.5
center = 5

m_a = 1 / (1 + np.abs((set_a - center) / width) ** slope)

plt.plot(set_a, m_a, 'g', label = 'Bell-shaped Membership
    Function')
plt.xlabel('Universe of Discourse')
plt.ylabel('Membership Degree')
```



AI-Generated

```
import numpy as np
import matplotlib.pyplot as plt

# --- Membership Functions ---

def triangular(x, a, b, c):
    """Triangular membership function with feet at a and c, peak
    at b."""
    return np.maximum(np.minimum((x - a) / (b - a), (c - x) / (c
    - b)), 0)

def trapezoidal(x, a, b, c, d):
    """Trapezoidal membership function with slopes from a to b
    and c to d."""
    return np.maximum(np.minimum(np.minimum((x - a)/(b - a), 1),
    (d - x)/(d - c)), 0)

def bell_shaped(x, a, b, c):
    """Generalized bell-shaped membership function centered at c.
    """
    return 1 / (1 + np.abs((x - c) / a) ** (2 * b))

# --- Universe of Discourse ---
x = np.linspace(0, 10, 500)

# --- Membership Function Values ---
tri_y = triangular(x, 2, 5, 8)
trap_y = trapezoidal(x, 2, 4, 6, 8)
bell_y = bell_shaped(x, a=1, b=4, c=5)

# --- Plotting ---
plt.figure(figsize=(10, 6))
```



```
plt.plot(x, tri_y, label="Triangular (a=2, b=5, c=8)", color='
blue')
plt.plot(x, trap_y, label="Trapezoidal (a=2, b=4, c=6, d=8)",
color='green')
plt.plot(x, bell_y, label="Bell-shaped (a=1, b=4, c=5)", color='
red')

plt.title("Triangular, Trapezoidal, and Bell-Shaped Membership
Functions")
plt.xlabel("x")
plt.ylabel("Membership Degree")
plt.ylim(-0.1, 1.1)
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```

Comments on comparisons

- The AI-generated code defines functions for triangular, trapezoidal, and bell-shaped membership functions, while my code directly plots the membership functions without defining separate functions.
- The AI code uses a more generalized approach to define the parameters of the membership functions, allowing for easy adjustments, while my code uses fixed parameters.
- Both codes use NumPy and Matplotlib for calculations and plotting.

Verified By

Cycle II

Experiment 9

Problem

Write a python program to fuzzify the input variable temperature using Triangular membership functions for Low, Medium, and High with Input temperature values [10, 20, 30, 40, 50].

Aim and Objectives

To fuzzify the input variable temperature using Triangular membership functions for Low, Medium, and High categories with specified input temperature values. Program Code

```
import skfuzzy as fuzzy
import numpy as np
import matplotlib.pyplot as plt
```

```
temperature = np.array([10, 20, 30, 40, 50])
temps = np.linspace(10, 50, 100)
```

```
low = fuzzy.trimf(temps, [10, 10, 30])
medium = fuzzy.trimf(temps, [20, 30, 40])
high = fuzzy.trimf(temps, [30, 50, 50])
```

```
plt.figure(figsize=(10, 5))
plt.plot(temps, low, label='Low', color='blue')
plt.plot(temps, medium, label='Medium', color='green')
plt.plot(temps, high, label='High', color='red')
plt.title('Fuzzy Membership Functions for Temperature')
plt.xlabel('Temperature (C)')
plt.ylabel('Membership Degree')
plt.legend()
plt.grid()
plt.show()
```

```
for temp in temperature:
    chance_low = fuzzy.interp_membership(temps, low, temp)
    chance_medium = fuzzy.interp_membership(temps, medium, temp)
    chance_high = fuzzy.interp_membership(temps, high, temp)

    chances = {
        'Low': chance_low,
        'Medium': chance_medium,
        'High': chance_high
    }

    max_chance = max(chances, key=chances.get)

    print(f"Temperature {temp} C: is a {max_chance}")
```

AI-Generated

```
import numpy as np
import skfuzzy as fuzz
import matplotlib.pyplot as plt

# Step 1: Define temperature universe
temperature = np.array([10, 20, 30, 40, 50])

# Step 2: Create a finer universe for smooth plotting and
          calculation
x_temp = np.linspace(10, 50, 100)

# Step 3: Define triangular membership functions
low = fuzz.trimf(x_temp, [10, 10, 30])      # Peak at 10, falls
          off by 30
medium = fuzz.trimf(x_temp, [20, 30, 40])    # Symmetric around 30
high = fuzz.trimf(x_temp, [30, 50, 50])      # Rises from 30, peak
          at 50

# Step 4: Fuzzify the input temperature values
print("Fuzzified outputs:")
for temp in temperature:
    low_val = fuzz.interp_membership(x_temp, low, temp)
    med_val = fuzz.interp_membership(x_temp, medium, temp)
    high_val = fuzz.interp_membership(x_temp, high, temp)
    print(f"Temperature: {temp} -> Low: {low_val:.2f}, Medium: {
        med_val:.2f}, High: {high_val:.2f}")

# Step 5: Plotting the membership functions
plt.figure(figsize=(10, 5))
plt.plot(x_temp, low, label='Low', color='blue')
plt.plot(x_temp, medium, label='Medium', color='green')
plt.plot(x_temp, high, label='High', color='red')

# Plot the input points on the curves
for temp in temperature:
    plt.axvline(temp, color='gray', linestyle='--', alpha=0.3)

plt.title("Triangular Membership Functions for Temperature")
plt.xlabel("Temperature")
plt.ylabel("Membership Degree")
plt.legend()
plt.grid(True)
plt.show()
```

Comments on comparisons

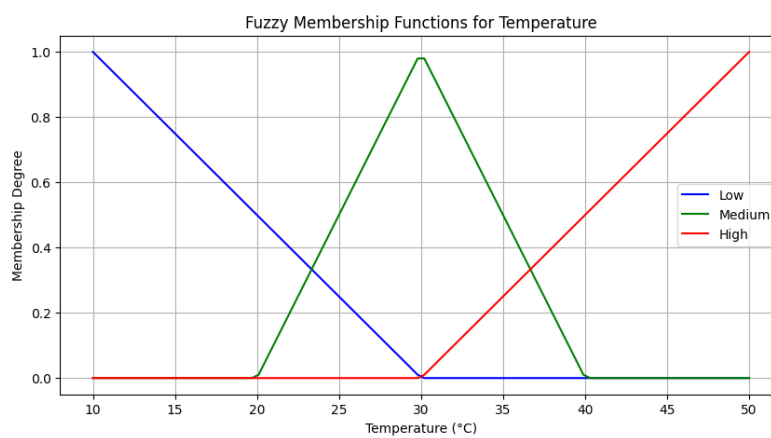
- The AI-generated code uses the 'skfuzzy' library to create triangular membership functions for Low, Medium, and High temperature categories, similar to my code.
- Both codes define the temperature range and fuzzify the input variable using trian-

gular membership functions.

- The AI code uses ‘interp_membership’ to calculate the membership degree for each temperature value and displays the probabilities for a temperature to be in all three categories.

Results

```
Temperature 10 C: is a Low
Temperature 20 C: is a Low
Temperature 30 C: is a Medium
Temperature 40 C: is a High
Temperature 50 C: is a High
```



Verified By

Cycle II

Experiment 10

Problem

Implement and compare defuzzification methods using the fuzzy output set. 10: 0.0, 20: 0.3, 30: 0.5, 40: 0.8, 50: 1.0, 60: 0.8, 70: 0.4, 80: 0.2. Visualize the fuzzy output and highlight each defuzzified result on the plot.

Aim and Objectives

To understand and implement defuzzification methods using a given fuzzy output set, and to visualize the fuzzy output while highlighting each defuzzified result on the plot.

Program Code

```
import skfuzzy as fuzzy
import matplotlib.pyplot as plt
import numpy as np
```

```
x = np.arange(10, 90, 10)
```

```
y = np.array([0.0, 0.3, 0.5, 0.8, 1.0, 0.8, 0.4, 0.2])
```

We can defuzzify the above fuzzy set in 6 ways

1. Centroid of Area (centroid)
2. Bisector of Area (bisector)
3. Mean of Maximum (mom)
4. Min of Maximum (som)
5. Max of Maximum (lom)

```
methods = ['centroid', 'bisector', 'mom', 'som', 'lom']
colors = ['r', 'g', 'b', 'c', 'm']

plt.figure(figsize=(10, 6))
for method, color in zip(methods, colors):
    center = fuzzy.defuzz(x, y, method)
    print(f"Defuzzification method: {method}, Center: {center:.2f}")
    plt.plot(x, y, color=color, label=f'{method} ({center:.2f})')
    plt.axvline(center, color=color, linestyle='--')
plt.title('Fuzzy Defuzzification Methods')
plt.xlabel('Input')
plt.ylabel('Output')
plt.legend()
plt.grid()
plt.show()
```

AI-Generated

```

import numpy as np
import matplotlib.pyplot as plt
import skfuzzy as fuzz

# Step 1: Define the fuzzy output set
x = np.array([10, 20, 30, 40, 50, 60, 70, 80])
membership = np.array([0.0, 0.3, 0.5, 0.8, 1.0, 0.8, 0.4, 0.2])

# Step 2: Interpolate to a finer universe for better
defuzzification
x_fine = np.linspace(10, 80, 1000)
membership_fine = np.interp(x_fine, x, membership)

# Step 3: Apply defuzzification methods
centroid = fuzz.defuzz(x_fine, membership_fine, 'centroid')
bisector = fuzz.defuzz(x_fine, membership_fine, 'bisector')
mom = fuzz.defuzz(x_fine, membership_fine, 'mom')
som = fuzz.defuzz(x_fine, membership_fine, 'som')
lom = fuzz.defuzz(x_fine, membership_fine, 'lom')

# Step 4: Print results
print("Defuzzification Results:")
print(f"Centroid: {centroid:.2f}")
print(f"Bisector: {bisector:.2f}")
print(f"Mean of Maximum (MoM): {mom:.2f}")
print(f"Smallest of Maximum (SoM): {som:.2f}")
print(f"Largest of Maximum (LoM): {lom:.2f}")

# Step 5: Visualization
plt.figure(figsize=(12, 6))
plt.plot(x, membership, 'bo', label='Fuzzy Output Points')
plt.plot(x_fine, membership_fine, 'b-', alpha=0.6, label='
    Interpolated Fuzzy Set')

# Plot defuzzified values
plt.axvline(centroid, color='r', linestyle='--', label=f'Centroid
    : {centroid:.2f}')
plt.axvline(bisector, color='g', linestyle='--', label=f'Bisector
    : {bisector:.2f}')
plt.axvline(mom, color='purple', linestyle='--', label=f'MoM: {
    mom:.2f}')
plt.axvline(som, color='orange', linestyle='--', label=f'SoM: {
    som:.2f}')
plt.axvline(lom, color='brown', linestyle='--', label=f'LoM: {lom
    :.2f}')

plt.title("Defuzzification of Fuzzy Output Set")
plt.xlabel("Output Variable (e.g., Temperature)")
plt.ylabel("Membership Degree")
plt.grid(True)
plt.legend()

```

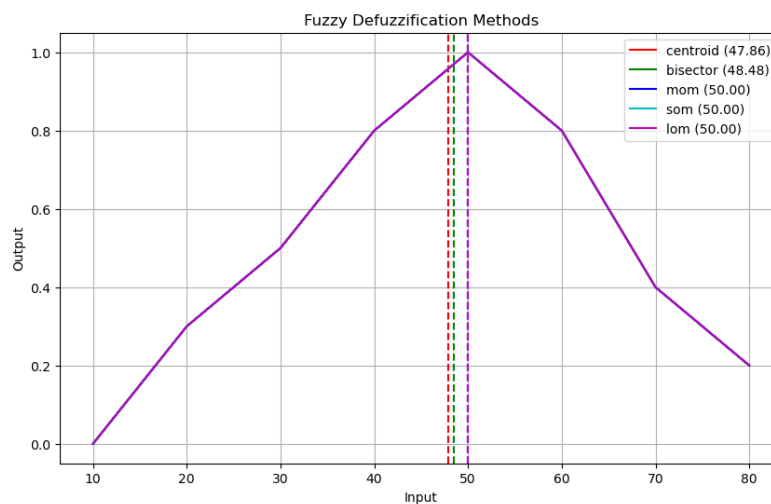
```
plt.show()
```

Comments on comparisons

- The AI-generated code uses the 'skfuzzy' library to implement defuzzification methods, similar to my code.
- Both codes define a fuzzy output set and visualize the defuzzification methods on a plot.

Results

```
Defuzzification method: centroid, Center: 47.86
Defuzzification method: bisector, Center: 48.48
Defuzzification method: mom, Center: 50.00
Defuzzification method: som, Center: 50.00
Defuzzification method: lom, Center: 50.00
```



Verified By

Cycle II

Experiment 11

Problem

Implement a Python script to model a Markov Decision Process using a given transition matrix and reward function of 3 states and 2 actions. Calculate the value function for each state provided the policy for all states and actions are initialized as 0.5. Provide a sample MDP and demonstrate the corresponding value function (Set discount factor = 0.9).

Aim and Objectives

To understand and implement Markov Decision Processes (MDPs) using a transition matrix and reward function, and to calculate the value function for each state given a policy initialized as 0.5.

Program Code

```
import matplotlib.pyplot as plt
import numpy as np
```

```
states = [0, 1, 2]
actions = [0, 1]
discount_factor = 0.9

policy = {0: {0: 0.5, 1: 0.5},
          1: {0: 0.5, 1: 0.5},
          2: {0: 0.5, 1: 0.5}}

transitions = {
    0: {
        0: [(0.6, 0, 5)],
        1: [(0.4, 2, 10)]
    },
    1: {
        0: [(0.5, 1, 3)],
        1: [(0.5, 2, 4)]
    },
    2: {
        0: [(0.7, 0, -5)],
        1: [(0.3, 1, 2)]
    }
}
```

```
def policy_evaluation(states, actions, policy, transitions,
                     discount_factor, theta=1e-6):
    V = {s: 0.0 for s in states}
    while True:
        delta = 0
        for s in states:
```



```

        v = 0
        for a in actions:
            action_prob = policy[s][a]
            for (prob, next_state, reward) in transitions[s][a]:
                v += action_prob * prob * (reward +
                    discount_factor * V[next_state])
            delta = max(delta, abs(v - V[s]))
        V[s] = v
    if delta < theta:
        break
return V

```

```

v = policy_evaluation(states, actions, policy, transitions,
    discount_factor)

```

```

print("The value function for each state is:")
for state in states:
    print(f"State {state}: {v[state]:.2f}")

```

AI-Generated

```

num_states = 3
num_actions = 2
gamma = 0.9 # Discount factor

T = np.array([
    [[0.8, 0.2, 0.0], [0.1, 0.6, 0.3]], # Transitions from state 0
    [[0.0, 0.9, 0.1], [0.4, 0.0, 0.6]], # Transitions from state 1
    [[0.2, 0.4, 0.4], [0.3, 0.3, 0.4]] # Transitions from state 2
])

R = np.array([
    [5, 10], # Rewards for actions in state 0
    [-1, 2], # Rewards for actions in state 1
    [0, 4] # Rewards for actions in state 2
])

policy = np.full((num_states, num_actions), 0.5)

V = np.zeros(num_states)

def evaluate_policy(policy, T, R, gamma, theta=1e-6):
    V = np.zeros(num_states)
    while True:
        delta = 0
        for s in range(num_states):
            v = 0

```

```

        for a in range(num\actions):
            action_prob = policy[s][a]
            reward = R[s][a]
            transition_probs = T[s][a]
            v += action_prob * (reward + gamma * np.dot(
                transition_probs, V))
            delta = max(delta, abs(v - V[s]))
            V[s] = v
        if delta < theta:
            break
    return V

V = evaluate_policy(policy, T, R, gamma)

for s in range(num_states):
    print(f"Value of state {s}: {V[s]:.4f}")

```

Comments on comparisons

- The AI-generated code uses NumPy to define the transition matrix and reward function, similar to my code.
- Both codes implement a policy evaluation function to calculate the value function for each state.
- The AI code structures the transition matrix and reward function in a more compact way, while my code uses dictionaries for better readability.

Results

```

The value function for each state is:
State 0: 4.90
State 1: 2.38
State 2: 0.41

```

Verified By